

Содержание

1	Сеточное решение уравнения Пуассона	4
1.1	Постановка задачи	4
1.1.1	Граничные условия первого рода	4
1.1.2	Граничные условия второго рода	4
1.1.3	Граничные условия третьего рода	4
1.1.3.1	Об универсальности условий третьего рода	5
1.1.4	Периодические граничные условия	5
1.2	Метод конечных разностей	6
1.2.1	Аппроксимация во внутренней области	6
1.2.2	Сборка СЛАУ	7
1.2.3	Учёт граничных условий	8
1.2.3.1	Граничные условия первого рода	8
1.2.3.2	Граничные условия второго рода	8
1.2.3.3	Граничные условия третьего рода	9
1.2.3.4	Периодические граничные условия	9
1.2.4	Верификация порядка аппроксимации	10
A	Задания для самостоятельной работы	12
A.1	Лекция 2 (12.09.26) МКР для решения уравнения Пуассона	13
B	Детали программной реализации	16
B.1	Программная реализация	17
B.1.1	Функция верхнего уровня	17
B.1.2	Детали реализации	18
C	Формулы и обозначения	21
C.1	Векторы	22
C.1.1	Обозначение	22
C.1.2	Набла–нотация	22
C.2	Интегрирование	24
C.2.1	Формула Гаусса–Остроградского	24
C.2.2	Интегрирование по частям	24
C.2.3	Численное интегрирование в заданной области	25
C.3	Интерполяционные полиномы	26
C.3.1	Многочлен Лагранжа	26
C.3.1.1	Узловые базисные функции	26
C.3.1.2	Интерполяция в параметрическом отрезке	27
C.3.1.3	Интерполяция в параметрическом треугольнике	30
C.3.1.4	Интерполяция в параметрическом квадрате	32

D	Алгоритмы	35
D.1	Геометрические алгоритмы	36
D.1.1	Линейная интерполяция	36
D.1.2	Преобразование координат	36
D.1.2.1	Матрица Якоби	37
D.1.2.2	Дифференцирование в параметрической плоскости	38
D.1.2.3	Интегрирование в параметрической плоскости	39
D.1.2.4	Двумерное линейное преобразование. Параметрический треугольник	39
D.1.2.5	Двумерное билинейное преобразование. Параметрический квадрат	40
D.1.2.6	Трёхмерное линейное преобразование. Параметрический тетраэдр	40
D.1.3	Свойства многоугольника	40
D.1.3.1	Площадь многоугольника	40
D.1.3.2	Интеграл по многоугольнику	42
D.1.3.3	Центр масс многоугольника	43
D.1.4	Свойства многогранника	43
D.1.4.1	Объём многогранника	43
D.1.4.2	Интеграл по многограннику	43
D.1.4.3	Центр масс многогранника	43
D.1.5	Поиск многоугольника, содержащего заданную точку	44
D.2	Форматы хранения разреженных матриц	45
D.2.1	CSR-формат	45
D.2.2	Массив словарей	47
D.3	Методы решения систем уравнений	49
D.3.1	Простые итерационные алгоритмы	49
D.3.1.1	Метод Якоби	49
D.3.1.2	Метод Зейделя	49
D.3.1.3	Метод последовательных верхних релаксаций SOR	50
D.3.2	Метод коррекции поправки	50
D.3.3	Методы подпространств Крылова	51
D.3.3.1	Итерационный процесс BiCGStab с предобуславливателем	51
E	Работа с инфраструктурой проекта CFDCourse	53
E.1	Клонирование	54
E.2	Разворачивание контейнера	55
E.3	Базовая разработка	56
E.3.1	Особенности проекта	56
E.3.2	Сборка в отладочном режиме	56
E.3.3	Сборка в релизном режиме	57
E.3.4	Работа с кодом	57
E.4	Разработка в vscode	57
E.4.1	Подключение к контейнеру	57
E.4.2	Настройки vscode	58

E.4.3	Сборка и отладка	59
E.5	Работа с системой контроля версий	60
E.5.1	Порядок работы с репозиторием CFDCourse	60
E.5.1.1	Получение последнего коммита	60
E.5.1.2	Создание коммита с текущим дз	60
E.6	Paraview	62
E.6.1	Данные на одномерных сетках	62
E.6.2	Изолинии для двумерного поля	65
E.6.3	Данные на двумерных сетках в виде поверхности	66
E.6.4	Числовых значения в точках и ячейках	67
E.6.5	Векторные поля	68
E.6.6	Значение функции вдоль линии	70
E.7	Hybmesh	72
E.7.1	Построение сеток	72
E.8	Приёмы отладки	73
E.8.1	Печать разреженной матрицы	73
E.8.2	Сохранение отладочных полей решения	73
E.8.3	Профилирование	74

1 Сеточное решение уравнения Пуассона

1.1 Постановка задачи

Будем рассматривать многомерное дифференциальное уравнение в области Ω :

$$-\nabla \cdot (\lambda(\mathbf{x}) \nabla u) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega \quad (1.1)$$

Оператор в левой части (оператор Лапласа) описывает физический процесс диффузии с коэффициентом диффузии λ . Это уравнение (с нулевой правой частью) используют в частности для расчёта распределения температуры в однородном твёрдом теле. В этом случае коэффициент λ называют коэффициентом теплопроводности, а за счёт ненулевой f можно задавать дополнительные внутренние источники тепла.

В простом случае постоянного коэффициента диффузии ($\lambda = \text{const}$) его можно вынести из под дивергенции и отнести в правую часть. Тогда уравнение упростится до однородного вида:

$$-\nabla^2 u = f(\mathbf{x}), \quad \mathbf{x} \in \Omega \quad (1.2)$$

Далее на границе области расчёта $\partial\Omega$ рассмотрим несколько типов граничных условий.

1.1.1 Граничные условия первого рода

Также известны как граничные условия Дирихле. На границе $\partial\Omega_I$ задано точное значение искомой функции u^Γ :

$$u = u^\Gamma(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_I \quad (1.3)$$

В аналогии задачи теплопроводности это условие можно трактовать как условие заданной на стенке температуры.

1.1.2 Граничные условия второго рода

Также известны как граничные условия Неймана. На границе $\partial\Omega_{II}$ задано значение нормальной производной искомой функции q :

$$-\lambda(\mathbf{x}) \frac{\partial u}{\partial n} = q(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_{II} \quad (1.4)$$

В аналогии задачи теплопроводности это условие можно трактовать как условие заданного на стенке теплового потока.

1.1.3 Граничные условия третьего рода

Также известны как граничные условия Робэна. На границе $\partial\Omega_{III}$ задано линейное соотношение значений функции и нормальной производной:

$$-\lambda(\mathbf{x}) \frac{\partial u}{\partial n} = \alpha(\mathbf{x})u + \beta(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_{III}. \quad (1.5)$$

При постановке задачи теплопроводности это условие часто записывают в виде

$$-\lambda(\mathbf{x}) \frac{\partial u}{\partial n} = \alpha(\mathbf{x}) (u - u^0).$$

где известное значение u^0 называют температурой окружающей среды. Такое условие называют условием конвективной теплопроводности или условием Ньютона–Рихмана. Для приведения этого условия к исходному виду (1.5) достаточно положить $\beta = -\alpha u^0$.

1.1.3.1 Об универсальности условий третьего рода

Условия (1.3), (1.4) можно свести к условиям (1.5) при правильном подборе коэффициентов α и β . Так для условий второго рода нужно положить $\alpha = 0$, $\beta = q$. А для условий первого рода: $\alpha = \varepsilon^{-1}$, $\beta = -\alpha u^\Gamma$, где $\varepsilon \rightarrow 0$ – некоторое очень малое положительное число.

1.1.4 Периодические граничные условия

Необходимость в таких условиях возникает при расчёте физических процессов около периодических структур: решёток, лопастей, рядов скважин, оребрения нагревателя и т.п. В этом случае из исходную большую область расчёта представляют как бесконечную последовательность однотипных ячеек периодичности, в каждой из которых решения полностью идентичны (в более сложных вариантах – сдвинуты на константу).

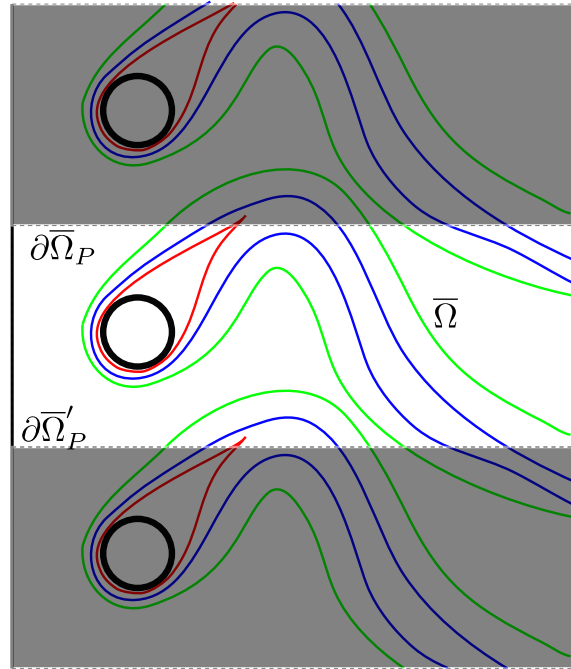


Рис. 1: Ячейка периодичности в задаче обтекания бесконечной решётки

На рис. 1 представлен пример области с выделенной ячейкой периодичности $\bar{\Omega}$ (незатенённая область). Изолинии можно трактовать как изотермы решения задачи о нестационарном обтекании решётки нагревателя (поле температур в этом случае описывается более сложным уравнением, чем (1.1) и приведено тут только для иллюстрации периодичности).

Пара периодических границ обозначена через $\partial\bar{\Omega}_P$ и $\partial\bar{\Omega}'_P$. Пусть эти границы топологически эквивалентны, то есть для любой точки $\mathbf{x} \in \partial\bar{\Omega}_P$ существует взаимнооднозначная точка $\mathbf{x}' \in$

$\partial\bar{\Omega}'_P$. Для того, чтобы решение за этими границами точно соответствовало решению внутри ячейки периодичности необходимо задать равенство значений и производных любого порядка:

$$\begin{cases} u(\mathbf{x}) = u(\mathbf{x}'), \\ \left. \frac{\partial^k u}{\partial n^k} \right|_{\mathbf{x}} = - \left. \frac{\partial^k u}{\partial n^k} \right|_{\mathbf{x}'} \end{cases} \quad \mathbf{x} \in \partial\bar{\Omega}_P, \quad \mathbf{x}' \in \partial\bar{\Omega}'_P, \quad \forall k. \quad (1.6)$$

Здесь под n подразумевается внешняя к ячейке периодичности нормаль, поэтому в правой части условия для производных стоит минус.

1.2 Метод конечных разностей

Будем рассматривать задачу (1.2), (1.3) в упрощённой одномерной постановке:

$$-\frac{\partial^2 u}{\partial x^2} = f(x) \quad (1.7)$$

в области $x \in [a, b]$.

Для аппроксимации производных методом конечных разностей будем использовать разложение Тейлора:

$$u(x+h) = u(x) + hu'(x) + \frac{h^2}{2}u''(x) + \dots + \frac{h^k}{k!}u^{(k)}(x) \quad (1.8)$$

из которой можно получить приближение первого (направленная разность) и второго (симметричная разность) порядков:

$$u'(x) = \frac{u(x+h) - u(x)}{h} - \underbrace{\frac{h}{2}u''(x) - \frac{h^2}{6}u'''(x) - \dots}_{O(h)}, \quad (1.9)$$

$$u'(x) = \frac{u(x+h/2) - u(x-h/2)}{h} - \underbrace{\frac{h^2}{24}u'''(x) - \frac{h^4}{1920}u^{(5)}(x) - \dots}_{O(h^2)}. \quad (1.10)$$

1.2.1 Аппроксимация во внутренней области

В области решения $[a, b]$ введём равномерную сетку из N ячеек. Шаг сетки будет равен $h = (b-a)/N$. Узлы сетки запишем в виде сеточного вектора $\{x_i\}$ длины $N+1$, где $i = \overline{0, N}$. Определим сеточный вектор $\{u_i\}$ неизвестных, элементы которого определяют значение искомого численного решения в i -ом узле сетки.

Для аппроксимации оператора Лапласа из левой части исходного уравнения (1.7) сначала воспользуемся симметричной разностью (1.10) для снятия первой производной:

$$u''(x) = \frac{u'(x+h/2) - u'(x-h/2)}{h} + O(h^2). \quad (1.11)$$

Далее распишем первые производную с помощью той же формулы (1.10):

$$\begin{aligned} u'(x + h/2) &= \frac{u(x + h) - u(x)}{h} - \frac{h^2}{24}u'''(x + h/2) + O(h^4) \\ u'(x - h/2) &= \frac{u(x) - u(x - h)}{h} - \frac{h^2}{24}u'''(x + h/2) + O(h^4). \end{aligned}$$

Запишем их разность, учитывая

$$u'''(x + h/2) - u'''(x - h/2) = hu^{(4)}(x) + O(h^2).$$

Получим

$$u'(x + h/2) - u'(x - h/2) = \frac{u(x + h) - 2u(x) + u(x - h)}{h} - \frac{h^3}{24}u^{(4)}(x) + O(h^4).$$

Подставим последнее выражение в (1.12):

$$u''(x) = -\frac{-u(x + h) + 2u(x) - u(x - h)}{h^2} + O(h^2). \quad (1.12)$$

Перенесём это выражение на равномерную сетку и окончательно запишем разностную схему второго порядка для уравнения (1.7)

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i, \quad i = \overline{1, N-1}. \quad (1.13)$$

Здесь $\{f_i\}$ – известный сеточный вектор, определяемый через известную аналитическую функцию $f(x)$ в правой части уравнения (1.7) как

$$f_i = f(x_i). \quad (1.14)$$

1.2.2 Сборка СЛАУ

Линейные уравнения (1.13) относительно неизвестных u_i составляют систему вида

$$\sum_{j=0}^N A_{ij} u_j = b_i, \quad i = \overline{1, N-1}$$

с матричными коэффициентами

$$A_{ij} = \begin{cases} 2/h^2, & i = \overline{1, N-1}, j = i; \\ -1/h^2, & i = \overline{1, N-1}, j = i - 1; \\ -1/h^2, & i = \overline{1, N-1}, j = i + 1; \\ 0, & \text{иначе.} \end{cases} \quad (1.15)$$

и правой частью

$$b_i = f_i, \quad i = \overline{1, N-1}. \quad (1.16)$$

Всего в этой системе $N - 1$ уравнение для $N + 1$ неизвестной. Для получения однозначного ответа не хватает двух уравнений, которые должны быть сформулированы из граничных условий (для правой и левой границ).

1.2.3 Учёт граничных условий

1.2.3.1 Граничные условия первого рода

Рассмотрим условия первого рода вида:

$$\begin{cases} u(a) = u_a, \\ u(b) = u_b. \end{cases} \quad (1.17)$$

Эти условия для сеточного вектора приобретут вид:

$$u_0 = u_a, \quad u_N = u_b.$$

В терминах матричных коэффициентов (1.15), (1.16) эти условия можно переписать в виде:

$$\begin{aligned} A_{00} &= 1, & b_0 &= u_a; \\ A_{NN} &= 1, & b_N &= u_b; \end{aligned} \quad (1.18)$$

1.2.3.2 Граничные условия второго рода

Рассмотрим условия Неймана

$$\begin{cases} -\frac{\partial u}{\partial n}\bigg|_a = q_a, \\ -\frac{\partial u}{\partial n}\bigg|_b = q_b, \end{cases} \quad (1.19)$$

Для их аппроксимации можно бы было воспользоваться направленными разностями (1.9). Учтём что нормаль $\mathbf{n}$ направлена наружу (против оси x на левой границе и по оси x – на правой). Тогда можно записать

$$-\frac{u_0 - u_1}{h} = q_a, \quad -\frac{u_N - u_{N-1}}{h} = q_b, \quad (1.20)$$

В терминах матричных коэффициентов получим

$$\begin{aligned} A_{00} &= 1/h & A_{01} &= -1/h & b_0 &= -q_a \\ A_{NN} &= 1/h & A_{N,N-1} &= -1/h & b_N &= -q_b \end{aligned} \quad (1.21)$$

Следует однако заметить, что формула (1.9) имеет первый порядок точности. Поэтому общий порядок расчётной схемы также будет первый. Чтобы аппроксимировать граничные условия второго рода вторым порядком воспользуемся уравнением (1.13) для аппроксимации оператора Лапласа в граничной (левой) точке:

$$u_0'' = \frac{u_{-1} - 2u_0 + u_1}{h^2}.$$

Здесь u_{-1} – это фиктивный узел, находящийся вне области расчёта. Для нахождения его значения

аппроксимируем граничное условие (1.24) с использованием симметричной схемы (1.10):

$$-\frac{u_{-1} - u_1}{2h} = q_a \quad \Rightarrow \quad u_{-1} = u_1 - 2hq_a.$$

Используя два последних соотношения запишем аппроксимированное уравнение Пуассона в граничной точке:

$$-u_0'' = f_0 \quad \Rightarrow \quad \frac{u_0 - u_1}{h^2} = -\frac{q_a}{h} + \frac{1}{2}f_0. \quad (1.22)$$

Переписывая в терминах матричных коэффициентов получим (сохраняя h^2 в знаменателе, чтобы выражение имело тот же порядок, что и выражение для внутренних узлов (1.13)):

$$\begin{aligned} A_{00} &= 1/h^2 & A_{01} &= -1/h^2 & b_0 &= -q_a/h + f_0/2 \\ A_{NN} &= 1/h^2 & A_{N,N-1} &= -1/h^2 & b_N &= -q_b/h + f_N/2 \end{aligned} \quad (1.23)$$

Ещё раз заметим, что все аппроксимационные формулы, использованные для получения (1.23) имели второй порядок точности, поэтому и само это выражение имеет второй порядок (в отличие от альтернативного (1.21)).

1.2.3.3 Граничные условия третьего рода

Аппроксимацию условий третьего рода

$$\begin{cases} -\left. \frac{\partial u}{\partial n} \right|_a = \alpha_a u(a) + \beta_a, \\ -\left. \frac{\partial u}{\partial n} \right|_b = \alpha_b u(b) + \beta_b, \end{cases} \quad (1.24)$$

можно получить по аналогии с аппроксимацией условий второго рода, заменив q на $\alpha u + \beta$. В частности, для левого узла аппроксимация второго порядка по аналогии с (1.22) примет вид

$$-u_0'' = \alpha_a u_0 + \beta \quad \Rightarrow \quad \frac{(1 + \alpha_a h)u_0 - u_1}{h^2} = -\frac{\beta}{h} + \frac{1}{2}f_0.$$

А на матричном уровне получим

$$\begin{aligned} A_{00} &= (1 + \alpha_a h)/h^2 & A_{01} &= -1/h^2 & b_0 &= -\beta_a/h + f_0/2 \\ A_{NN} &= (1 + \alpha_b h)/h^2 & A_{N,N-1} &= -1/h^2 & b_N &= -\beta_b/h + f_N/2 \end{aligned} \quad (1.25)$$

1.2.3.4 Периодические граничные условия

Для удовлетворения периодических условий можно на сеточном уровне закольцевать сетку, переименовав узел с индексом N в узел 0, узел $N + 1$ в 1 и т.п. В сеточном векторе тогда останется N неизвестных. Уравнения для крайних точек примут вид

$$\frac{-u_{N-1} + 2u_0 - u_1}{h^2} = f_0, \quad \frac{-u_{N-2} + 2u_{N-1} - u_0}{h^2} = f_{N-1}. \quad (1.26)$$

1.2.4 Верификация порядка аппроксимации

Порядок аппроксимации показывает скорость приближения численного решения к точному с уменьшением сетки. Поэтому для подтверждения порядка необходимо

- Знать точное решение,
- Уметь вычислять функционал (норму, $\|\cdot\|$), характеризующий отклонение точного решения от численного,
- Сделать несколько расчётов на сетках с разной N и заполнить таблицу $\|\{u_i - u^e(x_i)\}\|(N)$,
- На основе этой таблицы построить график в логарифмических осях и по углу наклона кривой сделать вывод о порядке аппроксимации.

Выберем произвольную функцию u^e (достаточно сильно изменяющуюся на целевом отрезке $[a, b]$).

Далее путём прямого вычисления определим параметры задачи f , u_a , u_b такие, для которых функция u^e является точным решением задачи (1.7), (1.17).

Зададимся числом разбиений N и решим задачу для выбранным параметрами. В результате определим сеточный вектор численного решения $\{u_i\}$.

В качестве нормы выберем стандартное отклонение. В интегральном виде для многомерной функции $y(\mathbf{x})$ в области $\mathbf{x} \in D$ оно имеет вид

$$\|y(\mathbf{x})\|_2 = \sqrt{\frac{1}{|D|} \int_D y(\mathbf{x})^2 d\mathbf{x}}. \quad (1.27)$$

Упрощая до одномерного случая

$$\|y(x)\|_2 = \sqrt{\frac{1}{b-a} \int_a^b y(x)^2 dx}.$$

Вычислим этот интеграл численно на введённой ранее равномерной сетке $\{x_i\}$:

$$\|\{y_i\}\|_2 = \sqrt{\frac{1}{b-a} \sum_{i=0}^N w_i y_i^2},$$

где $\{w_i\}$ – вес (или "площадь влияния") i -ого узла:

$$w_i = \begin{cases} h/2, & i = 0, N; \\ h, & i = \overline{1, N-1}, \end{cases}$$

такая что

$$\sum_{i=0}^N w_i = b - a.$$

Окончательно среднеквадратичная норма отклонения численного решения от точного запишется

в виде

$$||\{u_i - u^e(x_i)\}||_2 = \sqrt{\frac{1}{b-a} \sum_{i=0}^N w_i (u_i - u_i^e)^2}. \quad (1.28)$$

Пример программы, реализующей этот алгоритм смотри в п. [B.1](#).

А Задания для самостоятельной работы

А.1 Лекция 2 (12.09.26) МКР для решения уравнения Пуассона

В тесте `poisson1-fdm` из файла `poisson_fdm_test.cpp` реализовано решение одномерного уравнения Пуассона с граничными условиями первого рода (смотри теорию в п. 1.2, объяснение примера в п. В.1). Проводится расчёт на сгущающихся сетках с количеством ячеек от 10 до 1000 и рассчитываются среднеквадратичные нормы отклонения полученного численного решения от точного. Решения сохраняются в vtk-файлы `poisson1_fdm_n={}.vtk`. Отталкиваясь от этой реализации необходимо:

1. Нарисовать график, на котором показать второй порядок сходимости схемы. При этом увеличить диапазон рассматриваемых разбиений до того N , при котором поведение нормы сохраняет второй порядок сходимости. Проиллюстрировать численное решение для разных разбиений в сравнении с точным решением в Paraview.
2. Вместо условия первого рода использовать условие Неймана на левой границе расчётной области. Сравнить подходы первого (1.21) и второго (1.23) порядков, для каждого из которых нарисовать график сходимости.
3. Написать аналогичный тест, но для двумерной задачи Пуассона с граничными условиями первого рода. Проиллюстрировать решение в Paraview и показать второй порядок сходимости.

Рекомендации

- Свои программы следует оформлять в виде отдельных тестов (вместо того, чтобы модифицировать существующие). Желательно, после того как программа заработает, сразу оставить несколько базовых **ЧЕКС** проверок и сделать локальный коммит, чтобы впоследствии легко распознавать и исправлять внесённые в дальнейшей работе ошибки. К тому же наличие готовых тестов значительно облегчает рефакторинг кода.

При написании новых тестов следует переиспользовать уже написанный код, избегая копирования. Для этого необходимо оформлять повторяющийся код в виде отдельных процедур и пользоваться механизмами наследования классов.

- Для иллюстрации одномерного решения в Paraview для п.1 см. п. Е.6.1.
- Для программирования условий второго рода (п.2) вносить изменения нужно только в процедуры `Problem::approximate_lhs`, `Problem::approximate_rhs`. Вычислять граничное значение q_a следует используя имеющееся точное решение.
- Для выполнения п.3 необходимо написать новый тест, со своим классом-решателем `Problem2D`. Для уменьшения повторов в коде можно воспользоваться механизмом наследования классов. В этом классе нужно определить двумерное точное решение `exact_solution` и рассчитать соответствующую ему правую часть `rhs`.
- Двумерная задача Пуассона в частных производных будет иметь вид

$$-\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f(x, y).$$

Разностную схему для него удобно писать и использованием парного индекса

$$\frac{-u_{k[i-1,j]} + 2u_{k[i,j]} - u_{k[i+1,j]}}{h_x^2} + \frac{-u_{k[i,j-1]} + 2u_{k[i,j]} - u_{k[i,j+1]}}{h_y^2} = f_{k[i,j]},$$

где

$$k[i, j] = i + (n_x + 1)j \quad (\text{A.1})$$

– функция, переводящая парный (i, j) индекс узла регулярной сетки (i для оси x , j для оси y) в сквозной индекс k сеточного вектора, n_x – количество ячеек сетки в направлении x .

Функции, переводящие сквозной индекс в пару i, j , имеют вид

$$\begin{aligned} i[k] &= \text{mod}(k, (n_x + 1)), \quad // \text{остаток от деления,} \\ j[k] &= \lfloor k / (n_x + 1) \rfloor, \quad // \text{целая часть от деления.} \end{aligned} \quad (\text{A.2})$$

- Использовать класс `cfid::RegularGrid2D` для задания сетки. Функции перевода индексов узлов из сквозных в парные и обратно реализованы в классе двумерной регулярной сетки:

- `cfid::RegularGrid2D::to_split_point_index`
- `cfid::RegularGrid2D::to_linear_point_index`

- При вычислении весов w_k для вычисления среднеквадратичного отклонения учесть наличие граничных и угловых точек:

$$w_k = \begin{cases} h_x h_y / 4, & \text{для угловых точек;} \\ h_x h_y / 2, & \text{для граничных неугловых точек;} \\ h_x h_y, & \text{для внутренних точек.} \end{cases}$$

Четрые угловые точки определяются как

$$i[k], j[k] = (0, 0), (0, n_y), (n_x, n_y), (n_x, 0)$$

Граничные неугловые точки:

$$\begin{aligned} i[k], j[k] = \quad & \overline{1, n_x - 1}, 0; & \text{нижняя сторона,} \\ & n_x, \overline{1, n_y - 1}; & \text{правая сторона,} \\ & \overline{1, n_x - 1}, n_y; & \text{верхняя сторона,} \\ & 0, \overline{1, n_y - 1}; & \text{левая сторона.} \end{aligned}$$

- В случае, если решатель системы линейных уравнений не решает построенную матрицу, использовать функцию `cfid::dbg::print` для отладочной печати матрицы в консоль (размерность задачи должна быть небольшой).
- Для иллюстрации двумерного решения в Paraview использовать изолинии (E.6.2) и трёхмерные поверхности (E.6.3).

- Следует иметь ввиду, что на графике сходимости по оси абсцисс отложено линейное разбиение, вычисляемое как $n = 1/h$, где h – это характерный линейный размер ячейки. Для двумерных сеток этот линейный размер сетки можно вычислить через среднюю площадь ячейки A как $h = \sqrt{A}$, которую в свою очередь можно получить, разделив общую площадь на количество ячеек: $A = |D|/N$. Тогда, в случае единичного квадрата, линейное разбиение будет равно $n = \sqrt{N}$.

В Детали программной реализации

В.1 Программная реализация

Тестовая программа для решения одномерного уравнения Пуассона реализована в файле `poisson_fdm_test.cpp`.

В качестве аналитической тестовой функции используется

$$u^e = \sin(10x^2)$$

на отрезке $x \in [0, 1]$.

В.1.1 Функция верхнего уровня

объявлена как

```
122     std::cout << std::endl << "--- [poisson1-fdm] --- " << std::endl;
```

В программе в цикле по набору разбиений `n_cells`

```
134     // build test solver
```

создаётся решатель для тестовой задачи, использующий заданное число ячеек

```
136
```

вычисляется среднеквадратичная норма отклонения численного решения от точного

```
139
```

полученное численное решение (вместе с точным) сохраняется в vtk файле

```
poisson1_fdm_n={10,20,...}.vtk
```

```
142
```

а полученная норма печатается в консоль напротив количества ячеек

```
145
```

В результате работы программы в консоли должна отобразиться таблица вида

```
--- [poisson1] ---
10 0.179124
20 0.0407822
50 0.00634718
100 0.00158055
200 0.000394747
500 6.31421e-05
1000 1.57849e-05
```

где первый столбец – это количество ячеек, а второй – полученная для этого количества ячеек норма. Нарисовав график этой таблицы в логарифмических осях подтвердим второй порядок аппроксимации (рис. 2).

Открыв один из сохранённых в процессе работы файлов vtk `poisson1_fdm_n=?.vtk` в paraview можно посмотреть полученные графики. В файле представлены как точное “exact”, так и численное решение “numerical” (рис. 3).

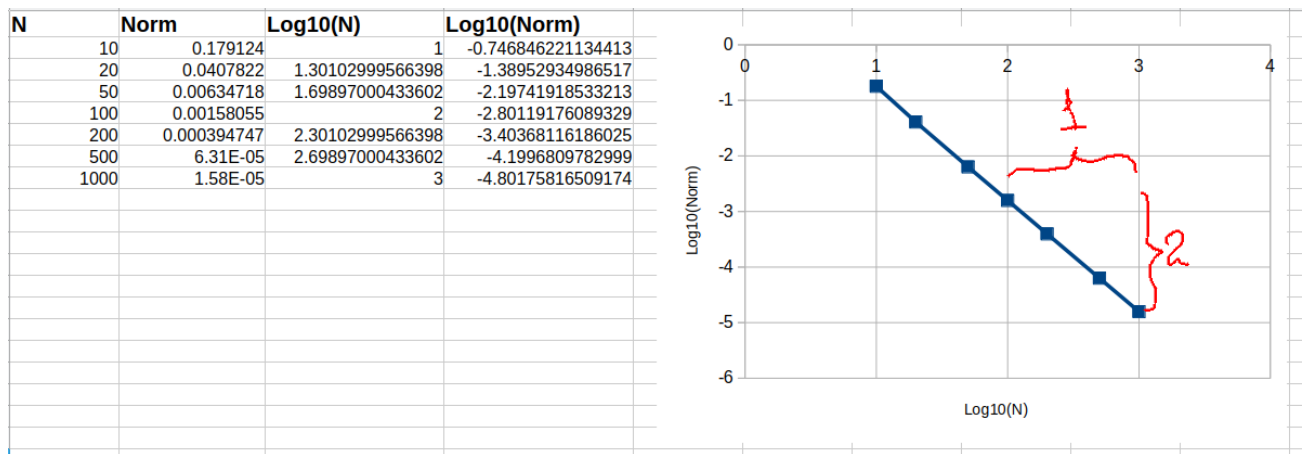


Рис. 2: Сходимость с уменьшением разбиения при решении одномерного уравнения Пуассона

B.1.2 Детали реализации

Основная работа по решению задачи проводится в классе `Problem`.

В его конструкторе происходит инициализация сетки (приватного поля класса) на отрезке $[0, 1]$ с заданным разбиением `n_cells`:

```
16 class Problem {
```

В методе

`solve()` производится численное решения задачи и вычисления нормы. Для этого последовательно

1. Строится матрица левой части и вектор правой части определяющей системы уравнений. Матрицы хранятся в разреженном формате CSR (п. D.2.1), удобном для последовательного чтения.
2. Вызывается решатель СЛАУ. Решение записывается в приватное поле класса `u`.
3. Вызывается функция вычисления нормы.

```
33 double solve() {
34     // 1. build SLAE
35     CsrMatrix mat = approximate_lhs();
36     std::vector<double> rhs = approximate_rhs();
37
38     // 2. solve SLAE
39     AmgcMatrixSolver solver;
40     solver.set_matrix(mat);
41     solver.solve(rhs, u_);
42
43     // 3. compute norm2
44     return compute_norm2();
45 }
```

Функции нижнего уровня (используемые в методе `solve`):

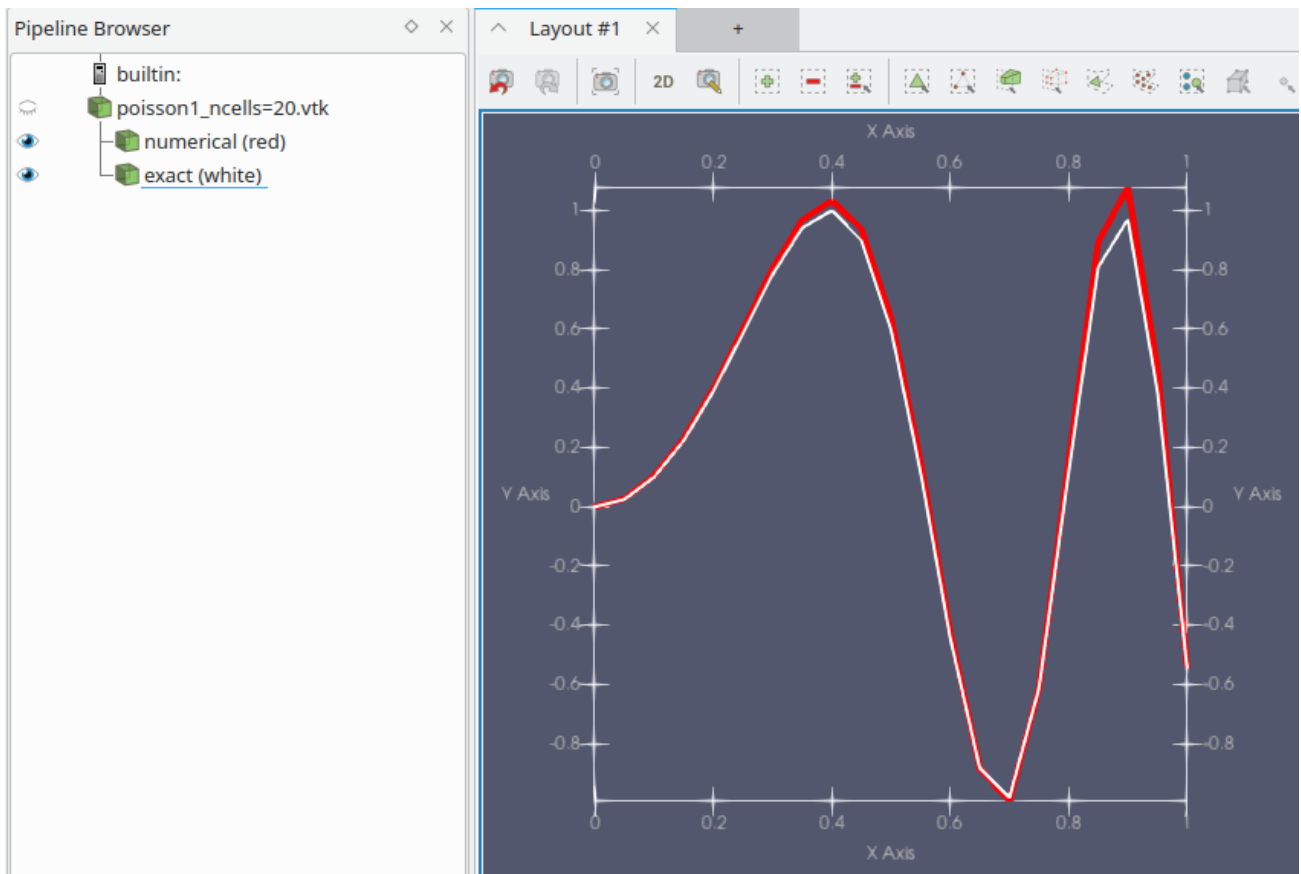


Рис. 3: Сравнение точного и численного решений уравнения Пуассона

- Сборка левой части СЛАУ. Реализует формулу (1.15). Для заполнения матрицы используется формат `cfdd::LodMatrix` (п. D.2.2), удобный для непоследовательной записи, который в конце конвертируется CSR.

```

67  CsrMatrix approximate_lhs() const {
68      // constant h = x[1] - x[0]
69      double h = grid_.point(1).x - grid_.point(0).x;
70
71      // fill using 'easy-to-construct' sparse matrix format
72      LodMatrix mat(grid_.n_points());
73      // internal
74      double diag = 2.0 / h / h;
75      double nondiag = -1.0 / h / h;
76      for (size_t i = 1; i < grid_.n_points() - 1; ++i) {
77          mat.add_value(i, i - 1, nondiag);
78          mat.add_value(i, i + 1, nondiag);
79          mat.add_value(i, i, diag);
80      }
81      // bc
82      mat.add_value(0, 0, 1);
83      mat.add_value(grid_.n_points() - 1, grid_.n_points() - 1, 1);
84

```

```

85     // return 'easy-to-use' sparse matrix format
86     return mat.to_csr();
87 }

```

- Сборка правой части СЛАУ. Реализует формулу (1.16).

```

89     std::vector<double> approximate_rhs() const {
90         std::vector<double> ret(grid_.n_points());
91         // internal
92         for (size_t i = 1; i < grid_.n_points() - 1; ++i) {
93             ret[i] = rhs(grid_.point(i));
94         }
95         // bc
96         ret[0] = exact_solution(grid_.point(0));
97         ret[grid_.n_points() - 1] = exact_solution(grid_.point(grid_.n_points() -
98             ↪ 1));
99
100         return ret;
101     }

```

- Вычисление нормы. Реализует формулу (1.28).

```

102     double compute_norm2() const {
103         // weights
104         double h = grid_.point(1).x - grid_.point(0).x;
105         std::vector<double> w(grid_.n_points(), h);
106         w[0] = w[grid_.n_points() - 1] = h / 2;
107
108         // sum
109         double sum = 0;
110         for (size_t i = 0; i < grid_.n_points(); ++i) {
111             double diff = u_[i] - exact_solution(grid_.point(i));
112             sum += w[i] * diff * diff;
113         }
114
115         double len = grid_.point(grid_.n_points() - 1).x - grid_.point(0).x;
116         return std::sqrt(sum / len);
117     }

```

С Формулы и обозначения

С.1 Векторы

С.1.1 Обозначение

Геометрические вектора обозначаются жирным шрифтом $\mathbf{v}$. Скалярные координаты вектора – через нижний индекс с обозначением оси координат: (v_x, v_y, v_z) . Если вектор $\mathbf{u}$ – вектор скорости, то его декартовы координаты имеют специальное обозначение $\mathbf{u} = (u, v, w)$. Единичные вектора, соответствующие осям координат, обозначаются знаком $\hat{\cdot}$: $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$. Координатные векторы обозначаются по символу первой оси. Например, $\mathbf{x} = (x, y, z)$ или $\boldsymbol{\xi} = (\xi, \eta, \zeta)$.

Операции в векторах имеют следующее обозначение (расписывая в декартовых координатах):

- Умножение на скалярную функцию

$$f\mathbf{u} = (fu_x)\hat{\mathbf{x}} + (fu_y)\hat{\mathbf{y}} + (fu_z)\hat{\mathbf{z}}; \quad (\text{C.1})$$

- Скалярное произведение

$$\mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y + u_z v_z; \quad (\text{C.2})$$

- Векторное произведение

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \hat{\mathbf{x}} & \hat{\mathbf{y}} & \hat{\mathbf{z}} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = (u_y v_z - u_z v_y) \hat{\mathbf{x}} - (u_x v_z - u_z v_x) \hat{\mathbf{y}} + (u_x v_y - u_y v_x) \hat{\mathbf{z}}. \quad (\text{C.3})$$

В двумерном случае можно считать, что $u_z = v_z = 0$. Тогда результатом векторного произведения согласно (C.3) будет вектор, направленный перпендикулярно плоскости xy :

$$\mathbf{u} \times \mathbf{v} = (u_x v_y - u_y v_x) \hat{\mathbf{z}}.$$

При работе с двумерными задачами, где ось $\mathbf{z}$ отсутствует, обычно результатом векторного произведения считают скаляр

$$2D : \mathbf{u} \times \mathbf{v} = u_x v_y - u_y v_x. \quad (\text{C.4})$$

Геометрический смысл этого скаляра: площадь параллелограмма, построенного на векторах $\mathbf{u}$ и $\mathbf{v}$.

С.1.2 Набла–нотация

Символ ∇ – есть псевдовектор, который выражает покоординатные производные. Для декартовой системы координат (x, y, z) он запишется в виде

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right).$$

В радиальной (r, ϕ, z) :

$$\nabla = \left(\frac{\partial}{\partial r}, \frac{1}{r} \frac{\partial}{\partial \phi}, \frac{\partial}{\partial z} \right).$$

В цилиндрической (r, θ, ϕ) :

$$\nabla = \left(\frac{\partial}{\partial r}, \frac{1}{r} \frac{\partial}{\partial \theta}, \frac{1}{r \sin \theta} \frac{\partial}{\partial \phi} \right).$$

Удобство записи дифференциальных выражений с использованием ∇ заключается в независимости записи от вида системы координат. Но если требуется обозначить производную по конкретной координате, то, по аналогии с обычными векторами, это делается через нижний индекс:

$$\nabla_n f = \frac{\partial f}{\partial n}.$$

Для этого символа справедливы все векторные операции, описанные ранее. Так, применение ∇ к скалярной функции аналогично умножению вектора на скаляр (C.1) (здесь и далее приводятся покомпонентные выражения для декартовой системы):

$$\nabla f = (\nabla_x f, \nabla_y f, \nabla_z f) = \frac{\partial f}{\partial x} \hat{\mathbf{x}} + \frac{\partial f}{\partial y} \hat{\mathbf{y}} + \frac{\partial f}{\partial z} \hat{\mathbf{z}}. \quad (\text{C.5})$$

Результатом этой операции является вектор.

Скалярное умножение ∇ на вектор $\mathbf{v}$ по аналогии с (C.2) – есть дивергенция:

$$\nabla \cdot \mathbf{v} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} + \frac{\partial v_z}{\partial z} \quad (\text{C.6})$$

результат которой – скалярная функция.

Двойное применение ∇ к скалярной функции – это оператор Лапласа:

$$\nabla \cdot \nabla f = \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2} \quad (\text{C.7})$$

Ротор – аналог векторного умножения (C.3):

$$\nabla \times \mathbf{v} = \begin{vmatrix} \hat{\mathbf{x}} & \hat{\mathbf{y}} & \hat{\mathbf{z}} \\ \nabla_x & \nabla_y & \nabla_z \\ v_x & v_y & v_z \end{vmatrix} = \left(\frac{\partial v_z}{\partial y} - \frac{\partial v_y}{\partial z} \right) \hat{\mathbf{x}} - \left(\frac{\partial v_z}{\partial x} - \frac{\partial v_x}{\partial z} \right) \hat{\mathbf{y}} + \left(\frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y} \right) \hat{\mathbf{z}}. \quad (\text{C.8})$$

С.2 Интегрирование

С.2.1 Формула Гаусса–Остроградского

Формула Гаусса–Остроградского, связывающая интегрирование по объёму E с интегрированием по границе этого объёма Γ , для векторного поля $\mathbf{v}$ имеет вид

$$\int_E \nabla \cdot \mathbf{v} d\mathbf{x} = \int_{\Gamma} v_n ds, \quad (\text{C.9})$$

где $\mathbf{n}$ – внешняя по отношению к области E нормаль. Смысл этой формулы можно проиллюстрировать на одномерном примере. Пусть одномерное векторное поле $v_x = f(x)$ на отрезке $E = [a, b]$ задано функцией, представленной на рис. 4. Разобьём область на $N = 3$ равномерных подобласти

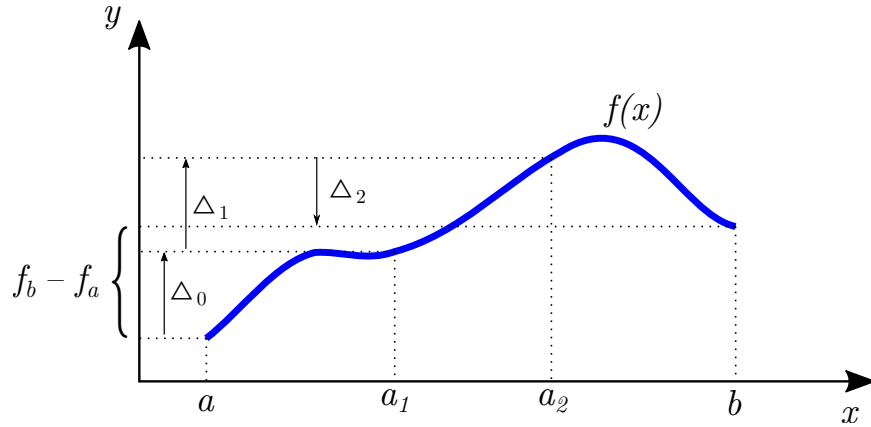


Рис. 4: Формула Гаусса-Остроградского в одномерном случае

длины h . Тогда расписывая интеграл как сумму, а производную через конечную разность, получим

$$\int_E \frac{\partial f}{\partial x} dx \approx \sum_{i=0}^2 h \left(\frac{\partial f}{\partial x} \right)_{i+\frac{1}{2}} \approx \sum_{i=0}^2 (f_{i+1} - f_i) = \Delta_0 + \Delta_1 + \Delta_2 = f_b - f_a.$$

Очевидно что, при устремлении $N \rightarrow \infty$ правая часть предыдущего выражения не изменится. То есть, сумма всех изменений функции в области есть изменение функции по её границам:

$$\int_a^b \frac{\partial f}{\partial x} dx = f(b) - f(a).$$

А формула (C.9) – есть многомерное обобщение этого выражения.

С.2.2 Интегрирование по частям

Подставив в (C.9) $\mathbf{v} = f\mathbf{u}$, где f – некоторая скалярная функция, и расписав дивергенцию в виде

$$\nabla \cdot (f\mathbf{u}) = f\nabla \mathbf{u} + \mathbf{u} \cdot \nabla f$$

получим формулу интегрирования по частям

$$\int_E \mathbf{u} \cdot \nabla f \, d\mathbf{x} = \int_{\Gamma} f u_n \, ds - \int_E f \nabla \cdot \mathbf{u} \, d\mathbf{x} \quad (\text{C.10})$$

Распишем некоторые частные случаи для формулы (C.10). Для $\mathbf{u} = (n_x, 0, 0)$ получим

$$\int_E \frac{\partial f}{\partial x} \, d\mathbf{x} = \int_{\Gamma} f \cos(\widehat{\mathbf{n}}, \mathbf{x}) \, ds \quad (\text{C.11})$$

При $\mathbf{u} = \nabla g$

$$\int_E f \nabla^2 g \, d\mathbf{x} = \int_{\Gamma} f \frac{\partial g}{\partial n} \, ds - \int_E \nabla f \cdot \nabla g \, d\mathbf{x} \quad (\text{C.12})$$

При $f = 1$ и $\mathbf{u} = \nabla g$

$$\int_E \nabla^2 g \, d\mathbf{x} = \int_{\Gamma} \frac{\partial g}{\partial n} \, ds \quad (\text{C.13})$$

C.2.3 Численное интегрирование в заданной области

Квадратурная формула

$$\int_E f(\mathbf{x}) \, d\mathbf{x} = \sum_{i=0}^{N-1} w_i f(\mathbf{x}_i) \quad (\text{C.14})$$

Она определяется заданием узлов интегрирования $\mathbf{x}_i$ и соответствующих весов w_i .

С.3 Интерполяционные полиномы

С.3.1 Многочлен Лагранжа

С.3.1.1 Узловые базисные функции

Рассмотрим функцию $f(\xi)$, заданную в области D . Внутри этой области зададим N узловых точек $\xi_i, i = \overline{0, N-1}$. Приближение функции f будем искать в виде

$$f(\xi) \approx \sum_{i=0}^{N-1} f_i \phi_i(\xi), \quad (\text{C.15})$$

где $f_i = f(\xi_i)$, ϕ_i – узловая базисная функция. Потребуем, чтобы это выражение выполнялось точно для всех заданных узлов интерполяции $\xi = \xi_i$. Тогда, исходя из определения (C.15), запишем условие на узловую базисную функцию

$$\phi_i(\xi_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases} \quad (\text{C.16})$$

Дополнительно потребуем, чтобы формула (C.15) была точной для постоянных функций

$$f(\xi) = \text{const} \quad \Rightarrow \quad f_i = \text{const}.$$

Тогда для любого ξ должно выполняться условие

$$\sum_{i=0}^{N-1} \phi_i(\xi) = 1, \quad \xi \in D. \quad (\text{C.17})$$

Задача построения интерполяционной функции состоит в конкретном определении узловых базисов $\phi_i(\xi)$ по заданному набору узловых точек ξ_i и значениям функции в них f_i . Будем искать базисы в виде многочленов вида

$$\phi_i(\xi) = \sum_a A_i^{(a)} \xi^a = A_i^{(0)} + A_i^{(1)} \xi + A_i^{(2)} \xi^2 + \dots, \quad i = \overline{0, N-1}. \quad (\text{C.18})$$

Определять коэффициенты $A_i^{(a)}$ будем из условий (C.16), которое даёт N линейных уравнений относительно неизвестных $A_i^{(a)}$ для каждого $i = \overline{0, N-1}$. Таким образом, в выражениях (C.18) должно быть ровно N слагаемых. Будем использовать последовательный набор степеней: $a = \overline{0, N-1}$. Выпишем систему линейных уравнений для 0-ой базисной функции

$$\begin{aligned} \phi_0(\xi_0) &= A_0^{(0)} + A_0^{(1)} \xi_0 + A_0^{(2)} \xi_0^2 + A_0^{(3)} \xi_0^3 + \dots = 1, \\ \phi_0(\xi_1) &= A_0^{(0)} + A_0^{(1)} \xi_1 + A_0^{(2)} \xi_1^2 + A_0^{(3)} \xi_1^3 + \dots = 0, \\ \phi_0(\xi_2) &= A_0^{(0)} + A_0^{(1)} \xi_2 + A_0^{(2)} \xi_2^2 + A_0^{(3)} \xi_2^3 + \dots = 0, \\ &\dots \end{aligned}$$

или в матричном виде

$$\begin{pmatrix} 1 & \xi_0 & \xi_0^2 & \xi_0^3 & \dots \\ 1 & \xi_1 & \xi_1^2 & \xi_1^3 & \dots \\ 1 & \xi_2 & \xi_2^2 & \xi_2^3 & \dots \\ 1 & \xi_3 & \xi_3^2 & \xi_3^3 & \dots \\ \dots & & & & \end{pmatrix} \begin{pmatrix} A_0^{(0)} \\ A_0^{(1)} \\ A_0^{(2)} \\ A_0^{(3)} \\ \vdots \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \end{pmatrix}$$

Записывая аналогичные выражения для остальных базисных функций, получим систему матричных уравнений вида $CA = E$:

$$\begin{pmatrix} 1 & \xi_0 & \xi_0^2 & \xi_0^3 & \dots \\ 1 & \xi_1 & \xi_1^2 & \xi_1^3 & \dots \\ 1 & \xi_2 & \xi_2^2 & \xi_2^3 & \dots \\ 1 & \xi_3 & \xi_3^2 & \xi_3^3 & \dots \\ \dots & & & & \end{pmatrix} \begin{pmatrix} A_0^{(0)} & A_1^{(0)} & A_2^{(0)} & A_3^{(0)} & \dots \\ A_0^{(1)} & A_1^{(1)} & A_2^{(1)} & A_3^{(1)} & \dots \\ A_0^{(2)} & A_1^{(2)} & A_2^{(2)} & A_3^{(2)} & \dots \\ A_0^{(3)} & A_1^{(3)} & A_2^{(3)} & A_3^{(3)} & \dots \\ \vdots & & & & \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & \dots \\ 0 & 1 & 0 & 0 & \dots \\ 0 & 0 & 1 & 0 & \dots \\ 0 & 0 & 0 & 1 & \dots \\ \vdots & & & & \end{pmatrix}$$

Отсюда матрица неизвестных коэффициентов A определится как

$$A = C^{-1} = \begin{pmatrix} 1 & \xi_0 & \xi_0^2 & \xi_0^3 & \dots \\ 1 & \xi_1 & \xi_1^2 & \xi_1^3 & \dots \\ 1 & \xi_2 & \xi_2^2 & \xi_2^3 & \dots \\ 1 & \xi_3 & \xi_3^2 & \xi_3^3 & \dots \\ \dots & & & & \end{pmatrix}^{-1}. \quad (\text{C.19})$$

Подставляя полином (C.18) в условие согласованности (C.17), получим требование

$$\sum_{i=0}^{N-1} A_i^{(a)} = \begin{cases} 1, & a = 0, \\ 0, & a = \overline{1, N-1}. \end{cases}$$

То есть сумма всех свободных членов в интерполяционных полиномах должна быть равна единице, а сумма коэффициентов при остальных степенях – нулю. Можно показать, что это свойство выполняется для любой матрицы $A = C^{-1}$, в случае, если первый столбец матрицы C состоит из единиц. То есть условие согласованности требует наличие свободного члена с интерполяционным полиномом.

С.3.1.2 Интерполяция в параметрическом отрезке

Будем рассматривать область интерполяции $D = [-1, 1]$. В качестве первых двух узлов интерполяции возьмем границы области: $\xi_0 = -1$, $\xi_1 = 1$.

Линейный базис Будем искать интерполяционный базис в виде

$$\phi_i(\xi) = A_i^{(0)} + A_i^{(1)}\xi.$$

на основе двух условий:

$$\phi_i(-1) = A_i^{(0)} - A_i^{(1)} = \delta_{0i}, \quad \phi_i(1) = A_i^{(0)} + A_i^{(1)} \delta_{1i}.$$

Составим матрицу C , записав эти условия в матричном виде

$$C = \left(\begin{array}{c|cc} & A^{(0)} & A^{(1)} \\ \hline \phi(-1) & 1 & -1 \\ \phi(1) & 1 & 1 \end{array} \right)$$

и, согласно (C.19), найдём матрицу коэффициентов

$$A = \begin{pmatrix} A_0^{(0)} & A_1^{(0)} \\ A_0^{(1)} & A_1^{(1)} \end{pmatrix} = C^{-1} = \begin{pmatrix} & \phi_0 & \phi_1 \\ \hline 1 & \frac{1}{2} & \frac{1}{2} \\ \xi & -\frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

Отсюда узловые базисные функции примут вид (рис. 5)

$$\begin{aligned} \phi_0(\xi) &= \frac{1-\xi}{2}, \\ \phi_1(\xi) &= \frac{1+\xi}{2}. \end{aligned} \tag{C.20}$$

Окончательно интерполяционная функция из определения (C.15) примет вид

$$f(\xi) \approx \frac{1-\xi}{2}f(-1) + \frac{1+\xi}{2}f(1).$$

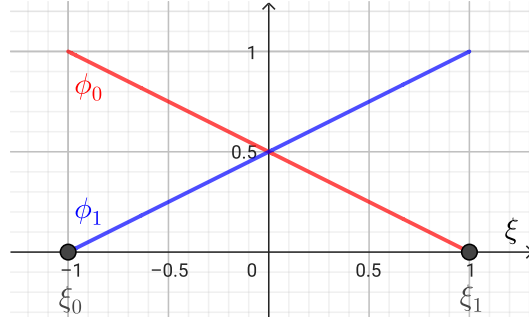


Рис. 5: Линейный базис в параметрическом отрезке

Квадратичный базис Будем искать интерполяционный базис в виде

$$\phi_i(\xi) = A_i^{(0)} + A_i^{(1)}\xi + A_i^{(2)}\xi^2.$$

По сравнению с линейным случаем, в форму базиса добавился ещё один неизвестный коэффициент $A_i^{(2)}$, поэтому в набор условий (C.16) требуется ещё одно уравнение (ещё одна узловая точка). Поче-

стим её в центр параметрического сегмента $\xi_2 = 0$. Далее будем действовать по аналогии с линейным случаем:

$$C = \left(\begin{array}{c|ccc} & A^{(0)} & A^{(1)} & A^{(2)} \\ \hline \phi(-1) & 1 & -1 & 1 \\ \phi(1) & 1 & 1 & 1 \\ \phi(0) & 1 & 0 & 0 \end{array} \right) \Rightarrow A = C^{-1} = \left(\begin{array}{c|ccc} & \phi_0 & \phi_1 & \phi_2 \\ \hline 1 & 0 & 0 & 1 \\ \xi & -\frac{1}{2} & \frac{1}{2} & 0 \\ \xi^2 & \frac{1}{2} & \frac{1}{2} & -1 \end{array} \right).$$

Узловые базисные функции для квадратичной интерполяции примут вид (рис. 6)

$$\begin{aligned} \phi_0(\xi) &= \frac{\xi^2 - \xi}{2}, \\ \phi_1(\xi) &= \frac{\xi^2 + \xi}{2}, \\ \phi_2(\xi) &= 1 - \xi^2. \end{aligned} \tag{C.21}$$

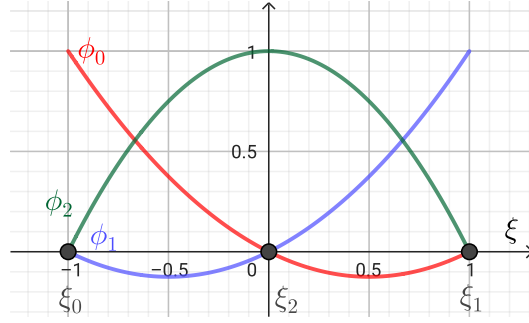


Рис. 6: Квадратичный базис в параметрическом отрезке

Кубический базис Интерполяционный базис будет иметь вид

$$\phi_i(\xi) = A_i^{(0)} + A_i^{(1)}\xi + A_i^{(2)}\xi^2 + A_i^{(3)}\xi^3.$$

Для нахождения четырёх коэффициентов нам понадобится четыре узла интерполяции. Две из них – это границы параметрического отрезка. Остальные две разместим так, чтобы разбить отрезок на равные интервалы: $\xi_2 = -\frac{1}{3}$, $\xi_3 = \frac{1}{3}$. Далее вычислим матрицу коэффициентов:

$$C = \left(\begin{array}{c|cccc} & A^{(0)} & A^{(1)} & A^{(2)} & A^{(3)} \\ \hline \phi(-1) & 1 & -1 & 1 & -1 \\ \phi(1) & 1 & 1 & 1 & 1 \\ \phi(-\frac{1}{3}) & 1 & -\frac{1}{3} & \frac{1}{9} & -\frac{1}{27} \\ \phi(\frac{1}{3}) & 1 & \frac{1}{3} & \frac{1}{9} & \frac{1}{27} \end{array} \right) \Rightarrow A = C^{-1} = \frac{1}{16} \left(\begin{array}{c|cccc} & \phi_0 & \phi_1 & \phi_2 & \phi_3 \\ \hline 1 & -1 & -1 & 9 & 9 \\ \xi & 1 & -1 & -27 & 27 \\ \xi^2 & 9 & 9 & -9 & -9 \\ \xi^3 & -9 & 9 & 27 & -27 \end{array} \right)$$

Узловые базисные функции для квадратичной интерполяции примут вид (рис. 7)

$$\begin{aligned}
 \phi_0(\xi) &= \frac{1}{16} (-1 + \xi + 9\xi^2 - 9\xi^3), \\
 \phi_1(\xi) &= \frac{1}{16} (-1 - \xi + 9\xi^2 + 9\xi^3), \\
 \phi_2(\xi) &= \frac{1}{16} (9 - 27\xi - 9\xi^2 + 27\xi^3), \\
 \phi_3(\xi) &= \frac{1}{16} (9 + 27\xi - 9\xi^2 - 27\xi^3),
 \end{aligned} \tag{C.22}$$

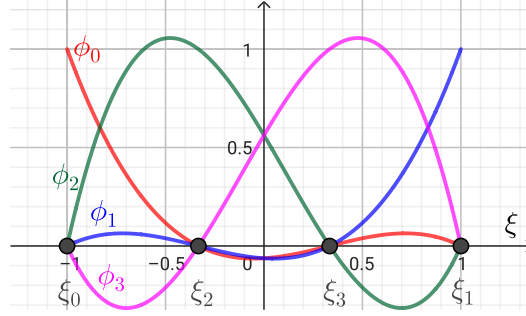


Рис. 7: Кубический базис в параметрическом отрезке

На рис. 8 представлено сравнение результатов аппроксимации функции $f(x) = -x + \sin(2x + 1)$ линейным, квадратичным и кубическим базисом. Видно, что все интерполяционные приближения точно попадают в функцию в своих узлах интерполяции, а между узлами происходит аппроксимация полиномом соответствующей степени.

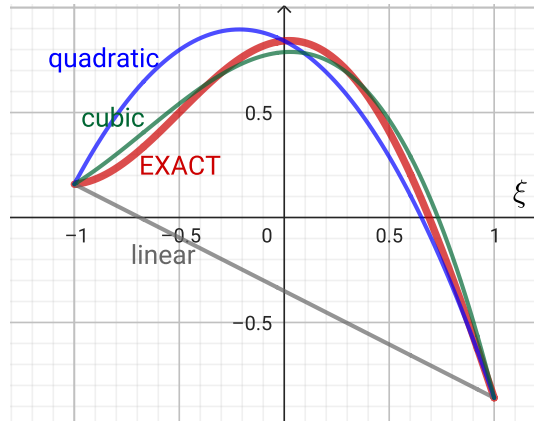


Рис. 8: Результат интерполяции

С.3.1.3 Интерполяция в параметрическом треугольнике

Теперь рассмотрим двумерное обобщение формулы

Линейный базис

$$\phi_i(\xi, \eta) = A_i^{(00)} + A_i^{(10)}\xi + A_i^{(01)}\eta.$$

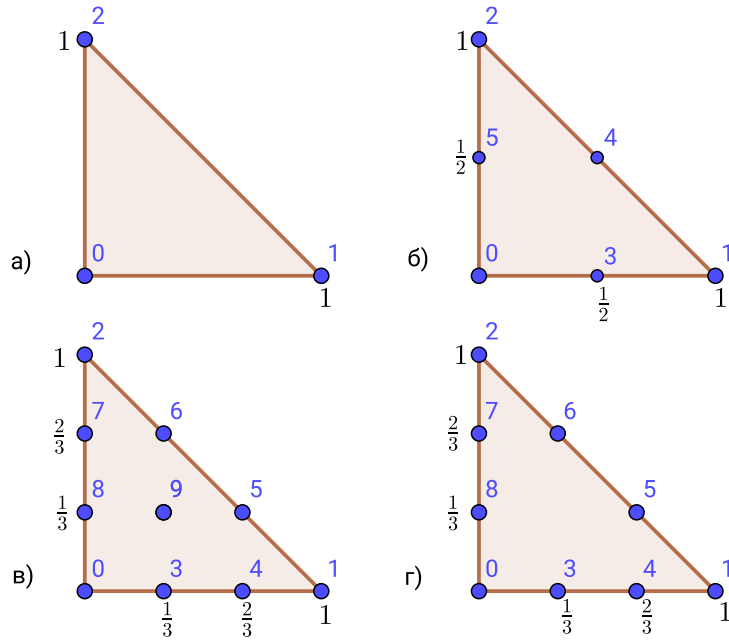


Рис. 9: Расположение узловых точек в параметрическом треугольнике. а) линейный базис, б) квадратичный базис, в) кубический базис, г) неполный кубический базис

$$C = \left(\begin{array}{c|ccc} & A^{(00)} & A^{(10)} & A^{(01)} \\ \hline \phi(0,0) & 1 & 0 & 0 \\ \phi(1,0) & 1 & 1 & 0 \\ \phi(0,1) & 1 & 0 & 1 \end{array} \right) \Rightarrow A = C^{-1} = \left(\begin{array}{c|ccc} & \phi_0 & \phi_1 & \phi_2 \\ \hline 1 & 1 & 0 & 0 \\ \xi & -1 & 1 & 0 \\ \eta & -1 & 0 & 1 \end{array} \right)$$

$$\phi_0(\xi, \eta) = 1 - \xi - \eta,$$

$$\phi_1(\xi, \eta) = \xi,$$

$$\phi_2(\xi, \eta) = \eta,$$

(C.23)

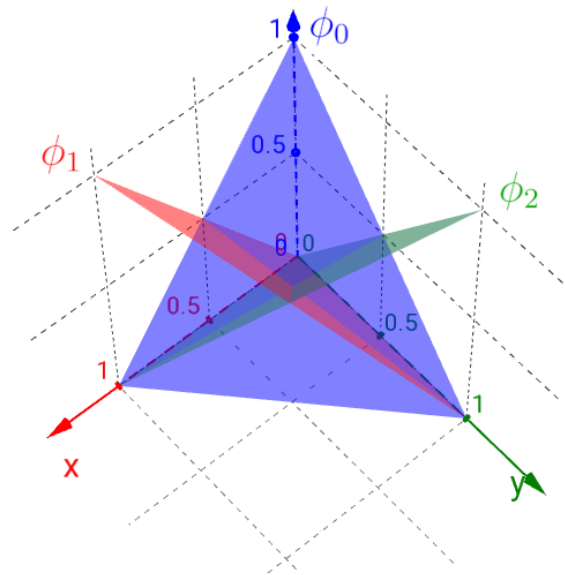


Рис. 10: Линейный базис в параметрическом треугольнике

Квадратичный базис

$$\phi_i(\xi, \eta) = A_i^{(00)} + A_i^{(10)}\xi + A_i^{(01)}\eta + A_i^{(11)}\xi\eta + A_i^{(20)}\xi^2 + A_i^{(02)}\eta^2.$$

$$C = \left(\begin{array}{c|cccccc} & A^{(00)} & A^{(10)} & A^{(01)} & A^{(11)} & A^{(20)} & A^{(02)} \\ \hline \phi(0,0) & 1 & 0 & 0 & 0 & 0 & 0 \\ \phi(1,0) & 1 & 1 & 0 & 0 & 1 & 0 \\ \phi(0,1) & 1 & 0 & 1 & 0 & 0 & 1 \\ \phi(\frac{1}{2},0) & 1 & \frac{1}{2} & 0 & 0 & \frac{1}{4} & 0 \\ \phi(\frac{1}{2},\frac{1}{2}) & 1 & \frac{1}{2} & \frac{1}{2} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \phi(0,\frac{1}{2}) & 1 & 0 & \frac{1}{2} & 0 & 0 & \frac{1}{4} \end{array} \right) \Rightarrow A = \left(\begin{array}{c|cccccc} & \phi_0 & \phi_1 & \phi_2 & \phi_3 & \phi_4 & \phi_5 \\ \hline 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ \xi & -3 & -1 & 0 & 4 & 0 & 0 \\ \eta & -3 & 0 & -1 & 0 & 0 & 4 \\ \xi\eta & 4 & 0 & 0 & -4 & 4 & -4 \\ \xi^2 & 2 & 2 & 0 & -4 & 0 & 0 \\ \eta^2 & 2 & 0 & 2 & 0 & 0 & -4 \end{array} \right)$$

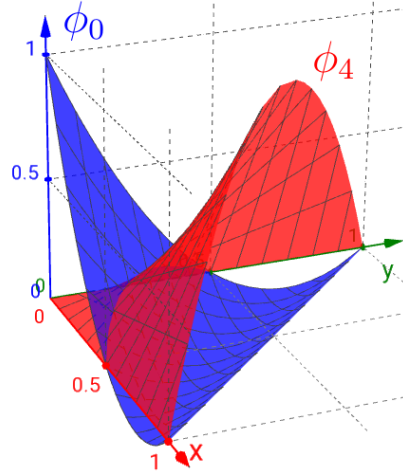


Рис. 11: Квадратичные функции ϕ_0, ϕ_4 в параметрическом треугольнике

Кубический базис TODO

Неполный кубический базис TODO

С.3.1.4 Интерполяция в параметрическом квадрате

Билинейный базис

$$\phi_i = A_i^{00} + A_i^{10}\xi + A_i^{01}\eta + A_i^{11}\xi\eta.$$

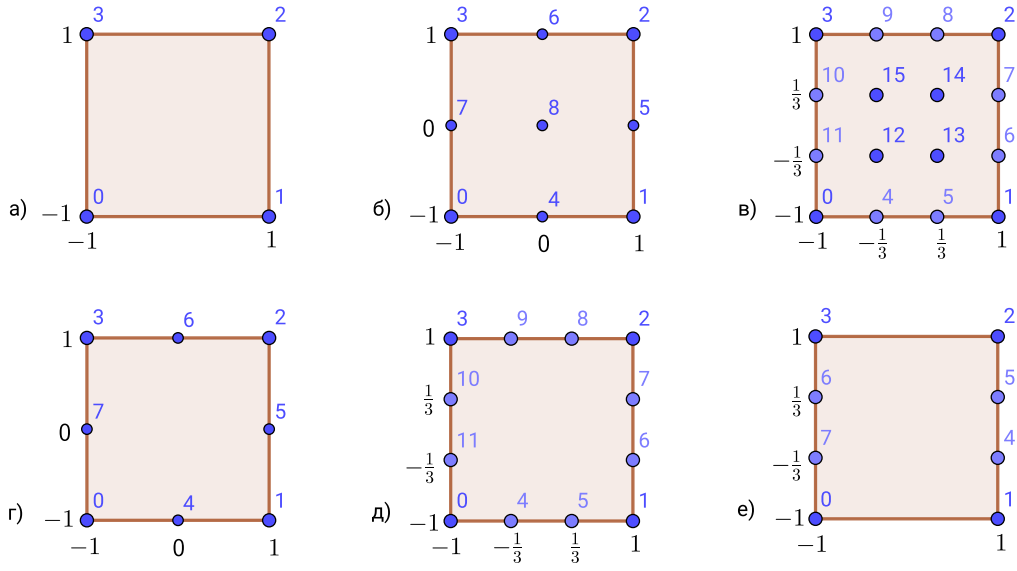


Рис. 12: Расположение узловых точек в параметрическом квадрате

$$C = \left(\begin{array}{c|cccc} & A^{(00)} & A^{(10)} & A^{(01)} & A^{(11)} \\ \hline \phi(-1, -1) & 1 & -1 & -1 & 1 \\ \phi(1, -1) & 1 & 1 & -1 & -1 \\ \phi(1, 1) & 1 & 1 & 1 & 1 \\ \phi(-1, 1) & 1 & -1 & 1 & -1 \end{array} \right) \Rightarrow A = C^{-1} = \frac{1}{4} \left(\begin{array}{c|cccc} & \phi_0 & \phi_1 & \phi_2 & \phi_3 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \xi & -1 & 1 & 1 & -1 \\ \eta & -1 & -1 & 1 & 1 \\ \xi\eta & 1 & -1 & 1 & -1 \end{array} \right)$$

$$\begin{aligned} \phi_0(\xi, \eta) &= \frac{1 - \xi - \eta + \xi\eta}{4} \\ \phi_1(\xi, \eta) &= \frac{1 + \xi - \eta - \xi\eta}{4} \\ \phi_2(\xi, \eta) &= \frac{1 + \xi + \eta + \xi\eta}{4} \\ \phi_3(\xi, \eta) &= \frac{1 - \xi + \eta - \xi\eta}{4} \end{aligned} \tag{C.24}$$

Определение двумерных базисов через комбинацию одномерных Обратим внимание, что в искомые билинейные базисные функции линейны в каждом из направлений ξ, η , если брать их по отдельности. Значит можно представить эти функции как комбинацию одномерных линейных базисов (C.20) в каждом из направлений. Узлы двумерного параметрического квадрата можно выразить через узлы линейного базиса в параметрическом одномерном сегменте, рассмотренном в п. C.3.1.2:

$$\xi_0 = (\xi_0^{1D}, \xi_0^{1D}), \quad \xi_1 = (\xi_1^{1D}, \xi_0^{1D}), \quad \xi_2 = (\xi_1^{1D}, \xi_1^{1D}), \quad \xi_3 = (\xi_0^{1D}, \xi_1^{1D}).$$

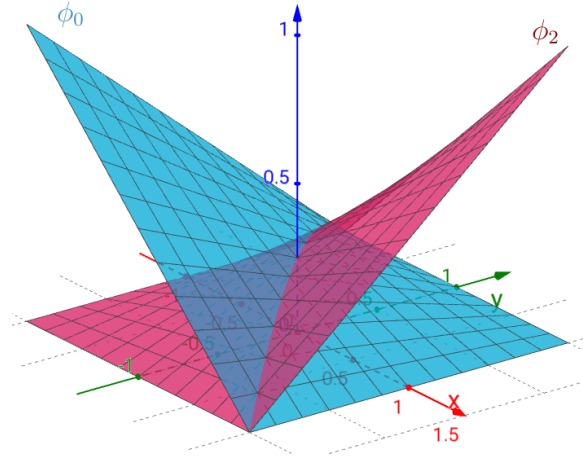


Рис. 13: Билинейные функции ϕ_0 , ϕ_2 в параметрическом квадрате

Значит и соответствующие базисные функции можно выразить через линейный одномерный базис ϕ^{1D} из соотношений (C.20):

$$\begin{aligned}\phi_0(\xi, \eta) &= \phi_0^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{1-\xi}{2}\frac{1-\eta}{2}, \\ \phi_1(\xi, \eta) &= \phi_1^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{1+\xi}{2}\frac{1-\eta}{2}, \\ \phi_2(\xi, \eta) &= \phi_1^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{1+\xi}{2}\frac{1+\eta}{2}, \\ \phi_3(\xi, \eta) &= \phi_0^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{1-\xi}{2}\frac{1+\eta}{2}.\end{aligned}$$

Раскрыв скобки можно убедиться, что мы получили тот же билинейный базис, что и ранее (C.24).

Биквадратичный базис Применим этот метод для вычисления биквадратичного базиса, определённого в точках на рис. 12б. В качестве основе возьмём квадратичный одномерный базис ϕ_i^{1D} из (C.21).

$$\begin{aligned}\phi_0(\xi, \eta) &= \phi_0^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{\xi^2 - \xi\eta^2 - \eta}{2}, & \phi_1(\xi, \eta) &= \phi_1^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{\xi^2 + \xi\eta^2 - \eta}{2}, \\ \phi_2(\xi, \eta) &= \phi_1^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{\xi^2 + \xi\eta^2 + \eta}{2}, & \phi_3(\xi, \eta) &= \phi_0^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{\xi^2 - \xi\eta^2 + \eta}{2}, \\ \phi_4(\xi, \eta) &= \phi_2^{1D}(\xi)\phi_0^{1D}(\eta) = (1 - \xi^2)\frac{\eta^2 - \eta}{2}, & \phi_5(\xi, \eta) &= \phi_1^{1D}(\xi)\phi_2^{1D}(\eta) = \frac{\xi^2 + \xi}{2}(1 - \eta^2), \\ \phi_6(\xi, \eta) &= \phi_2^{1D}(\xi)\phi_1^{1D}(\eta) = (1 - \xi^2)\frac{\eta^2 + \eta}{2}, & \phi_7(\xi, \eta) &= \phi_0^{1D}(\xi)\phi_2^{1D}(\eta) = \frac{\xi^2 - \xi}{2}(1 - \eta^2), \\ \phi_8(\xi, \eta) &= \phi_2^{1D}(\xi)\phi_2^{1D}(\eta) = (1 - \xi^2)(1 - \eta^2).\end{aligned}\tag{C.25}$$

Бикубический базис

Неполный биквадратичный базис

Неполный бикубический базис

D.1 Геометрические алгоритмы

D.1.1 Линейная интерполяция

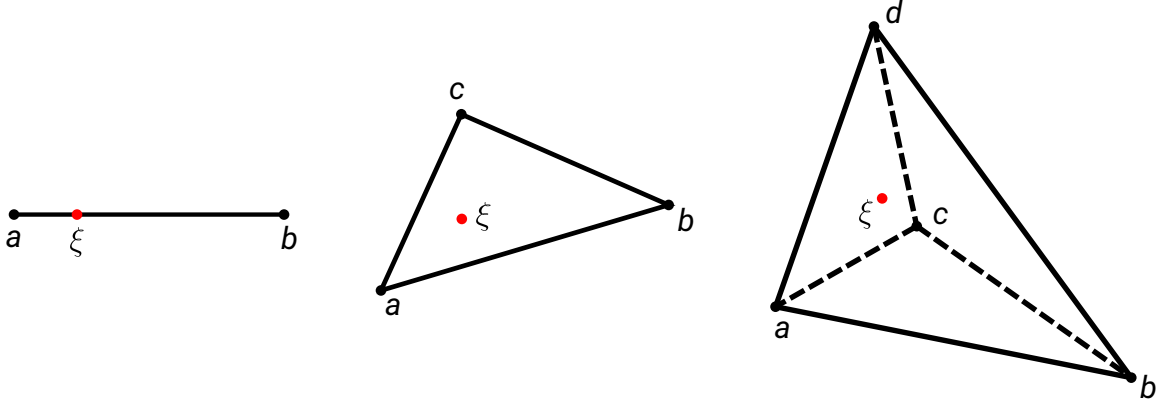


Рис. 14: Порядок нумерации точек одномерного, двумерного и трёхмерного симплекса при линейной интерполяции

Пусть функция u задана в узлах симплекса, имеющего нумерацию согласно рис. 14. Необходимо найти значение этой функции в точке ξ (эта точка вообще говоря не обязана лежать внутри симплекса).

Интерполяция в одномерном, двумерном и трёхмерном виде запишется как

$$u(\xi) = \frac{|\Delta_{\xi a}|u(b) + |\Delta_{b\xi}|u(a)}{|\Delta_{ba}|} \quad (\text{D.1})$$

$$u(\xi) = \frac{|\Delta_{ab\xi}|u(c) + |\Delta_{bc\xi}|u(a) + |\Delta_{ca\xi}|u(b)}{|\Delta_{abc}|} \quad (\text{D.2})$$

$$u(\xi) = \frac{|\Delta_{abc\xi}|u(d) + |\Delta_{cbd\xi}|u(a) + |\Delta_{cda\xi}|u(b) + |\Delta_{adb\xi}|u(c)}{|\Delta_{abcd}|}, \quad (\text{D.3})$$

где $|\Delta|$ – знаковый объём симплекса, вычисляемый как

$$|\Delta_{ab}| = b - a,$$

$$|\Delta_{abc}| = \left(\frac{(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})}{2} \right)_z,$$

$$|\Delta_{abcd}| = \frac{(\mathbf{b} - \mathbf{a}) \cdot ((\mathbf{c} - \mathbf{a}) \times (\mathbf{d} - \mathbf{a}))}{6}.$$

D.1.2 Преобразование координат

Рассмотрим преобразование из двумерной параметрической системы координат ξ в физическую систему $\mathbf{x}$. Такое преобразование полностью определяется покоординатными функциями $\mathbf{x}(\xi)$. Далее получим соотношения, связывающие операции дифференцирования и интегрирования в физической и параметрической областях.

D.1.2.1 Матрица Якоби

Будем рассматривать двумерное преобразование $(\xi, \eta) \rightarrow (x, y)$. Линеаризуем это преобразование (разложим в ряд Фурье до линейного слагаемого)

$$\begin{aligned} x(\xi_0 + d\xi, \eta_0 + d\eta) &\approx x_0 + \left. \frac{\partial x}{\partial \xi} \right|_{\xi_0, \eta_0} d\xi + \left. \frac{\partial x}{\partial \eta} \right|_{\xi_0, \eta_0} d\eta, \\ y(\xi_0 + d\xi, \eta_0 + d\eta) &\approx y_0 + \left. \frac{\partial y}{\partial \xi} \right|_{\xi_0, \eta_0} d\xi + \left. \frac{\partial y}{\partial \eta} \right|_{\xi_0, \eta_0} d\eta, \end{aligned}$$

где $x_0 = x(\xi_0, \eta_0)$, $y_0 = y(\xi_0, \eta_0)$. Переписывая это выражение в векторном виде, получим

$$\mathbf{x}(\xi_0 + d\xi) - \mathbf{x}_0 = J(\xi_0) d\xi. \quad (\text{D.4})$$

Матрица J (зависящая от точки приложения в параметрической плоскости) называется матрицей Якоби:

$$J = \begin{pmatrix} J_{11} & J_{12} \\ J_{21} & J_{22} \end{pmatrix} = \begin{pmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{pmatrix} \quad (\text{D.5})$$

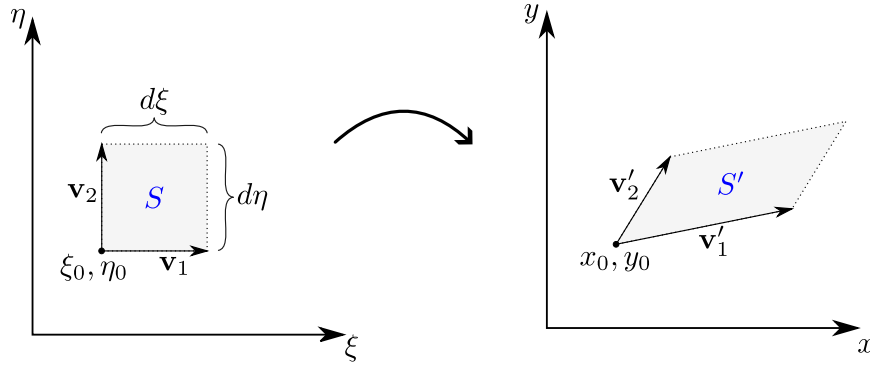


Рис. 15: Преобразование элементарного объёма

Якобиан Определитель матрицы Якоби (якобиан), взятый в конкретной точке параметрической плоскости ξ_0 , показывает, во сколько раз увеличился элементарный объём около этой точки в результате преобразования. Действительно, рассмотрим два перпендикулярных элементарных вектора в параметрической системе координат: $\mathbf{v}_1 = (d\xi, 0)$ и $\mathbf{v}_2 = (0, d\eta)$ отложенных от точки ξ_0 (см. рис. 15). В результате преобразования по формуле (D.4) получим следующие преобразования концевых точек и векторов:

$$\begin{aligned} (\xi_0, \eta_0) &\rightarrow (x_0, y_0), \\ (\xi_0 + d\xi, \eta_0) &\rightarrow (x_0 + J_{11}d\xi, y_0 + J_{21}d\xi) \Rightarrow \mathbf{v}_1 \rightarrow \mathbf{v}'_1 = (J_{11}d\xi, J_{21}d\xi), \\ (\xi_0, \eta_0 + d\eta) &\rightarrow (x_0 + J_{12}d\eta, y_0 + J_{22}d\eta) \Rightarrow \mathbf{v}_2 \rightarrow \mathbf{v}'_2 = (J_{12}d\eta, J_{22}d\eta). \end{aligned}$$

Элементарный объём равен площади параллелограмма, построенного на элементарных векторах. В параметрической плоскости согласно (C.4) получим

$$|S| = \mathbf{v}_1 \times \mathbf{v}_2 = d\xi d\eta,$$

и аналогично для физической плоскости:

$$|S'| = \mathbf{v}'_1 \times \mathbf{v}'_2 = (J_{11}J_{22} - J_{12}J_{21})d\xi d\eta = |J|d\xi d\eta$$

Сравнивая два последних соотношения приходим к выводу, что элементарный объём в результате преобразования увеличился в $|J|$ раз. Тогда можно записать

$$dx dy = |J| d\xi d\eta \quad (\text{D.6})$$

Многомерным обобщением этой формулы будет

$$d\mathbf{x} = |J| d\xi \quad (\text{D.7})$$

D.1.2.2 Дифференцирование в параметрической плоскости

Пусть задана некоторая функция $f(x, y)$. Распишем её производную по параметрическим координатам:

$$\begin{aligned} \frac{\partial f}{\partial \xi} &= \frac{\partial f}{\partial x} \frac{\partial x}{\partial \xi} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial \xi}, \\ \frac{\partial f}{\partial \eta} &= \frac{\partial f}{\partial x} \frac{\partial x}{\partial \eta} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial \eta}. \end{aligned}$$

Вспоминая определение (D.5), запишем

$$\begin{pmatrix} \frac{\partial f}{\partial \xi} \\ \frac{\partial f}{\partial \eta} \end{pmatrix} = J^T \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} J_{11} & J_{21} \\ J_{12} & J_{22} \end{pmatrix} \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix}$$

Обратная зависимость примет вид

$$\begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} = (J^T)^{-1} \begin{pmatrix} \frac{\partial f}{\partial \xi} \\ \frac{\partial f}{\partial \eta} \end{pmatrix} = \frac{1}{|J|} \begin{pmatrix} J_{22} & -J_{21} \\ -J_{12} & J_{11} \end{pmatrix} \begin{pmatrix} \frac{\partial f}{\partial \xi} \\ \frac{\partial f}{\partial \eta} \end{pmatrix}$$

В многомерном виде запишем

$$\nabla_{\mathbf{x}} f = (J^T)^{-1} \nabla_{\xi} f. \quad (\text{D.8})$$

D.1.2.3 Интегрирование в параметрической плоскости

Пусть в физической области $\mathbf{x}$ задана область D_x . Интеграл функции $f(\mathbf{x})$ по этой области можно расписать, используя замену (D.7)

$$\int_{D_x} f(\mathbf{x}) d\mathbf{x} = \int_{D_\xi} f(\xi) |J(\xi)| d\xi, \quad (\text{D.9})$$

где $f(\xi) = f(\mathbf{x}(\xi))$, а D_ξ – образ области D_x в параметрической плоскости.

D.1.2.4 Двумерное линейное преобразование. Параметрический треугольник

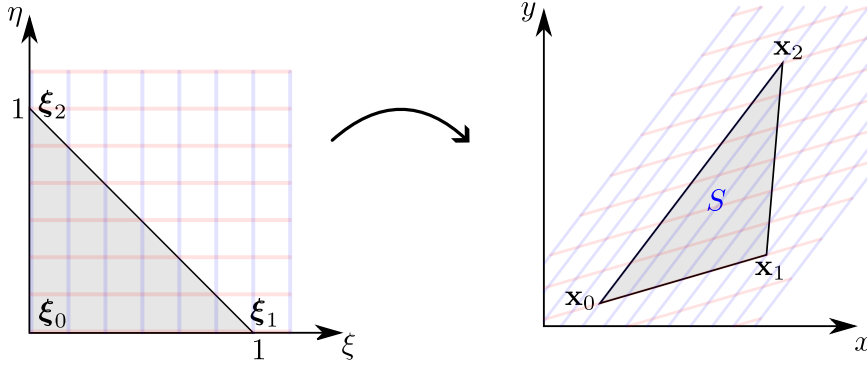


Рис. 16: Преобразование из параметрического треугольника

Рассмотрим двумерное преобразование, при котором определяющие функции являются линейными. То есть представимыми в виде

$$\begin{aligned} x(\xi, \eta) &= A_x \xi + B_x \eta + C_x, \\ y(\xi, \eta) &= A_y \xi + B_y \eta + C_y. \end{aligned}$$

Для определения шести констант, определяющих это преобразование, достаточно выбрать три любые (не лежащие на одной прямой) точки: $(\xi_i, \eta_i) \rightarrow (x_i, y_i)$ для $i = 0, 1, 2$. В результате получим систему из шести линейных уравнений (три точки по две координаты), из которой находятся константы $A_{x,y}, B_{x,y}, C_{x,y}$. Пусть три точки в параметрической плоскости образуют единичный прямоугольный треугольник (рис. 16):

$$\xi_0, \eta_0 = (0, 0), \quad \xi_1, \eta_1 = (1, 0), \quad \xi_2, \eta_2 = (0, 1).$$

Тогда система линейных уравнений примет вид

$$\begin{aligned} x_0 &= C_x, & y_0 &= C_y, \\ x_1 &= A_x + C_x, & y_1 &= A_y + C_y, \\ x_2 &= B_x + C_x, & y_2 &= B_y + C_y. \end{aligned}$$

Определив коэффициенты преобразования из этой системы, окончательно запишем преобразование

$$\begin{aligned} x(\xi, \eta) &= (x_1 - x_0)\xi + (x_2 - x_0)\eta + x_0, \\ y(\xi, \eta) &= (y_1 - y_0)\xi + (y_2 - y_0)\eta + y_0. \end{aligned} \quad (\text{D.10})$$

Матрица Якоби этого преобразования (D.5) не будет зависеть от параметрических координат ξ, η :

$$J = \begin{pmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{pmatrix}. \quad (\text{D.11})$$

Якобиан преобразования будет равен удвоенной площади треугольника S , составленного из определяющих точек в физической плоскости:

$$|J| = (x_1 - x_0)(y_2 - y_0) - (y_1 - y_0)(x_2 - x_0) = (\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0) = 2|S|. \quad (\text{D.12})$$

Распишем интеграл по треугольнику S по формуле (D.9). Вследствии линейности преобразования якобиан постоянен и, поэтому, его можно вынести его из-под интеграла:

$$\int_S f(x, y) dx dy = |J| \int_0^1 \int_0^{1-\xi} f(\xi, \eta) d\eta d\xi. \quad (\text{D.13})$$

D.1.2.5 Двумерное билинейное преобразование. Параметрический квадрат

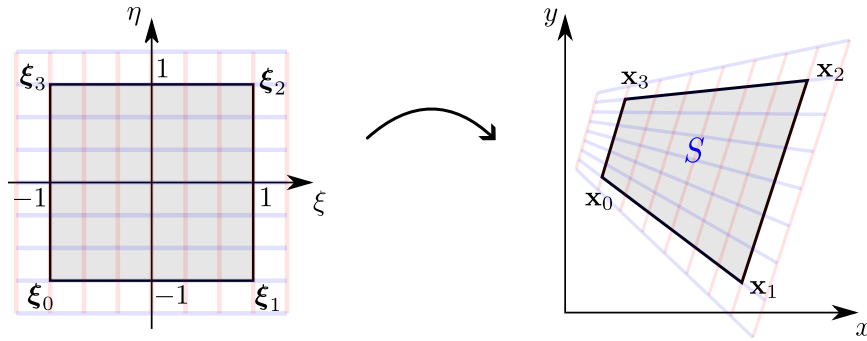


Рис. 17: Преобразование из параметрического квадрата

D.1.2.6 Трёхмерное линейное преобразование. Параметрический тетраэдр

TODO

D.1.3 Свойства многоугольника

D.1.3.1 Площадь многоугольника

Рассмотрим произвольный несамопересекающийся N -угольник S , заданный координатами своих узлов $\mathbf{x}_i$, $i = \overline{0, N-1}$, пронумерованных последовательно против часовой стрелки (рис. 18). Далее

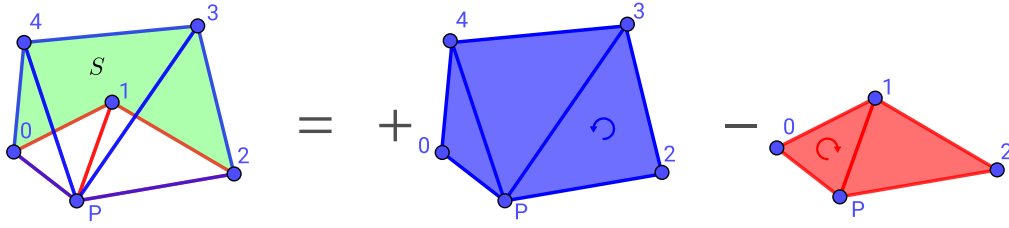


Рис. 18: Площадь произвольного многоугольника

введём произвольную точку $\mathbf{p}$ и от этой точки будем строить ориентированные треугольники до граней многоугольника:

$$\Delta_i^p = (\mathbf{p}, \mathbf{x}_i, \mathbf{x}_{i+1}), \quad i = \overline{0, N-1},$$

(для корректности записи будем считать, что $\mathbf{x}_N = \mathbf{x}_0$). Тогда площадь исходного многоугольника S будет равна сумме знаковых площадей треугольников Δ_i^p :

$$|S| = \sum_{i=0}^{N-1} |\Delta_i^p|, \quad |\Delta_i^p| = \frac{(\mathbf{x}_i - \mathbf{p}) \times (\mathbf{x}_{i+1} - \mathbf{p})}{2}.$$

Знак площади ориентированного треугольника зависит от направления закрутки его узлов: она положительна для закрутки против часовой стрелки и отрицательна, если узлы пронумерованы по часовой стрелке. В частности, на рисунке 18 видно, что треугольники, отмеченные красным: $P01, P12$, будут иметь отрицательную площадь, а синие треугольники $P23, P34, P40$ – положительную. Сумма этих площадей с учётом знака даст искомую площадь многоугольника.

Для сокращения вычислений воспользуемся произвольностью положения $\mathbf{p}$ и совместим её с точкой $\mathbf{x}_0$. Тогда треугольники $\Delta_0^p, \Delta_{N-1}^p$ вырождаются (будут иметь нулевую площадь). Обозначим такую последовательную триангуляцию как

$$\Delta_i = (\mathbf{x}_0, \mathbf{x}_i, \mathbf{x}_{i+1}), \quad i = \overline{1, N-2}. \quad (\text{D.14})$$

Знаковая площадь ориентированного треугольника будет равна

$$|\Delta_i| = \frac{(\mathbf{x}_i - \mathbf{x}_0) \times (\mathbf{x}_{i+1} - \mathbf{x}_0)}{2}. \quad (\text{D.15})$$

Тогда окончательно формула определения площади примет вид

$$|S| = \sum_{i=1}^{N-2} |\Delta_i|. \quad (\text{D.16})$$

Плоский полигон в пространстве Если плоский полигон S расположен в трёхмерном пространстве, то правая часть формулы (D.15) согласно определению векторного произведения в трёхмерном пространстве (C.3) – есть вектор. Чтобы получить скалярную площадь, нужно спроецировать этот вектор на единичную нормаль к плоскости многоугольника:

$$\mathbf{n} = \frac{\mathbf{k}}{|\mathbf{k}|}, \quad \mathbf{k} = (\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0).$$

Эта формула записана из предположения, что узел $\mathbf{x}_2$ не лежит на одной прямой с узлами $\mathbf{x}_0$, $\mathbf{x}_1$. Иначе вместо $\mathbf{x}_2$ нужно выбрать любой другой узел, удовлетворяющий этому условию. Тогда площадь ориентированного треугольника, построенного в трёхмерном пространстве запишется через смешанное произведение:

$$|\triangle_i| = \frac{((\mathbf{x}_i - \mathbf{x}_0) \times (\mathbf{x}_{i+1} - \mathbf{x}_0)) \cdot \mathbf{n}}{2}. \quad (\text{D.17})$$

Формула для определения площади полигона (D.16) будет по прежнему верна. При этом итоговый знак величины S будет положительным, если закрутка полигона положительная (против часовой стрелки) при взгляде со стороны вычисленной нормали $\mathbf{n}$.

D.1.3.2 Интеграл по многоугольнику

Рассмотрим интеграл функции $f(x, y)$ по N -угольнику S , заданному последовательными координатами своих узлов $\mathbf{x}_i$. Введём последовательную триангуляцию согласно (D.14). Тогда интеграл по многоугольнику можно расписать как сумму интегралов по ориентированным треугольникам:

$$\int_S f(x, y) dx dy = \sum_{i=1}^{N-2} \int_{\triangle_i} f(x, y) dx dy. \quad (\text{D.18})$$

Далее для вычисления интегралов в правой части воспользуемся преобразованием к параметрическому треугольнику (п. D.1.2.4). Следуя формуле интегрирования (D.13), распишем интеграл по i -ому треугольнику:

$$\int_{\triangle_i} f(x, y) dx dy = |J_i| \int_0^1 \int_0^{1-\xi} f_i(\xi, \eta) d\eta d\xi,$$

где якобиан $|J_i|$ согласно (D.12) есть удвоенная площадь ориентированного треугольника $\triangle_i$ (положительная при закрутке против часовой стрелке и отрицательная иначе):

$$|J_i| = 2|\triangle_i| = (\mathbf{x}_i - \mathbf{x}_0) \times (\mathbf{x}_{i+1} - \mathbf{x}_0),$$

а функция $f_i(\xi, \eta)$ есть функция от преобразованных согласно (D.10) переменных:

$$f_i(\xi, \eta) = f((\mathbf{x}_i - \mathbf{x}_0)\xi + (\mathbf{x}_{i+1} - \mathbf{x}_0)\eta + \mathbf{x}_0).$$

Окончательно запишем

$$\int_S f(x, y) dx dy = 2 \sum_{i=1}^{N-2} |\triangle_i| \int_0^1 \int_0^{1-\xi} f_i(\xi, \eta) d\eta d\xi. \quad (\text{D.19})$$

Отметим, что эта формула работает и в том случае, когда полигон расположен в трёхмерном пространстве (знаковую площадь при этом следует вычислять по (D.17)).

D.1.3.3 Центр масс многоугольника

По определению, координаты центра масс $\mathbf{c}$ области S равны среднеинтегральным значениям координатных функций. То есть

$$c_x = \frac{1}{|S|} \int_S x \, dx dy, \quad c_y = \frac{1}{|S|} \int_S y \, dx dy.$$

Далее распишем интеграл в правой части через последовательную триангуляцию согласно (D.18) с учётом линейного преобразования (D.10):

$$\begin{aligned} \int_S x \, dx dy &= \sum_{i=1}^{N-2} \int_{\Delta_i} x \, dx dy \\ &= \sum_{i=1}^{N-2} |J_i| \int_0^1 \int_0^{1-\xi} ((x_i - x_0)\xi + (x_{i+1} - x_0)\eta + x_0) \, d\eta d\xi \\ &= \sum_{i=1}^{N-2} \frac{|J_i|}{2} \frac{x_0 + x_i + x_{i+1}}{3} \\ &= \sum_{i=1}^{N-2} |\Delta_i| \frac{x_0 + x_i + x_{i+1}}{3}. \end{aligned}$$

Итого, с учётом (D.16), координаты центра масс примут вид

$$\mathbf{c} = \frac{\sum_{i=1}^{N-2} \frac{\mathbf{x}_0 + \mathbf{x}_i + \mathbf{x}_{i+1}}{3} |\Delta_i|}{\sum_{i=1}^{N-2} |\Delta_i|}.$$

Если полигон расположен в двумерном пространстве xu , то знаковая площадь треугольников вычисляется по формуле (D.15). В случае трёхмерного пространства должна использоваться формула (D.17).

D.1.4 Свойства многогранника

D.1.4.1 Объём многогранника

TODO

D.1.4.2 Интеграл по многограннику

TODO

D.1.4.3 Центр масс многогранника

TODO

D.1.5 Поиск многоугольника, содержащего заданную точку

TODO

D.2 Форматы хранения разреженных матриц

D.2.1 CSR-формат

При реализации решателей систем сеточных уравнений важно учитывать разреженный характер используемых в левой части матриц. То есть избегать хранения и ненужных операций с нулевыми элементами матрицы.

Здесь будем рассматривать общие форматы хранения, не привязанные к конкретному шаблону. Любой общий формат хранения должен хранить информацию о шаблоне матрице (адресах ненулевых элементов) и значениях матричных коэффициентов в этом шаблоне.

В CSR (Compressed sparse rows) формате все ненулевые элементы хранятся в линейном массиве `vals`. А шаблон матрицы – в двух массивах

- массиве колонок `cols` – значений колонок для соответствующих ему значений из массива `vals`,
- массиве адресов `addr` – индексах массива `vals`, с которых начинается описание соответствующей строки.

В конце массива `addr` добавляется общая длина массива `vals`.

Таким образом, длины массивов `vals`, `cols` равны количеству ненулевых элементов матрицы, а длина массива `addr` равна количеству строк в матрице плюс один.

Для облегчения процедур поиска описание каждой строки должно идти последовательно с увеличением индекса колонки.

Для примера рассмотрим следующую матрицу

$$\begin{pmatrix} 2.0 & 0 & 0 & 1.0 \\ 0 & 3.0 & 5.0 & 4.0 \\ 0 & 0 & 6.0 & 0 \\ 0 & 7.0 & 0 & 8.0 \end{pmatrix} \quad (\text{D.20})$$

Массивы, описывающие матрицу в формате CSR примут вид

	<i>row</i> = 0	<i>row</i> = 1	<i>row</i> = 2	<i>row</i> = 3	
<i>vals</i> =	2.0, 1.0,	3.0, 5.0, 4.0,	6.0,	7.0, 8.0	
<i>cols</i> =	0, 3,	1, 2, 3,	2,	1, 3	
<i>addr</i> =	0,	2,	5,	6,	8

Рассмотрим реализацию базовых алгоритмов для матриц, заданных в этом формате.

Пусть матрица задана следующими массивами:

```
std::vector<double> vals; // массив значений
std::vector<size_t> cols; // массив столбцов
std::vector<size_t> addr; // массив адресов
```

Число строк в матрице:

```
size_t nrows = addr.size()-1;
```

Число элементов в шаблоне (ненулевых элементов)

```
size_t n_nonzeros = vals.size();
```

Число ненулевых элементов в заданной строке 'irow'

```
size_t n_nonzeros_in_row = addr[irow+1] - addr[irow];
```

Умножение матрицы на вектор 'v' (длина этого вектора должна быть равна числу строк в матрице). Здесь реализуется суммирование вида

$$r_i = \sum_{j=0}^{N-1} a_{ij}v_j,$$

при этом избегаются лишние операции с нулями

```
// число строк в матрице и длина вектора v
size_t nrows = addr.size() - 1;
// массив ответов. Инициализируем нулями
std::vector<double> r(nrows, 0);
// цикл по строкам
for (size_t irow=0; irow < nrows; ++irow){
    // цикл по ненулевым элементам строки irow
    for (size_t a = addr[irow]; a < addr[irow+1]; ++a){
        // получаем индекс колонки
        size_t icol = cols[a];
        // значение матрицы на позиции [irow, icol]
        double val = vals[a];
        // добавляем к ответу
        r[irow] += val * v[icol];
    }
}
```

Поиск значения элемента матрицы по адресу (irow, icol) с учётом локально сортированного вектора cols

```
using iter_t = std::vector<size_t>::const_iterator;
// указатели на начало и конец описания строки в массиве cols
iter_t it_start = cols.begin() + addr[irow];
iter_t it_end = cols.begin() + addr[irow+1];
// поиск значения icol в отсортированной последовательности [it_start, it_end)
iter_t fnd = std::lower_bound(it_start, it_end, icol);
if (fnd != it_end && *fnd == icol){
    // если нашли, то определяем индекс найденного элемента в массиве cols
    size_t a = fnd - cols.begin();
    // и возвращаем значение из vals по этому индексу
}
```

```

return vals[a];
} else {
    // если не нашли, значит элемент [irow, icol] находится вне шаблона. Возвращаем 0
    return 0;
}

```

Формат CSR обеспечивает максимальную компактность хранения разреженной матрицы и при этом удобен для последовательной итерации по элементам матрицы (операции умножения матрицы на вектор), но его существенным недостатком является высокая сложность добавления нового элемента в шаблон.

D.2.2 Массив словарей

При реализации сеточных методов решения дифференциальных уравнений работу с матрицами можно разбить на два этапа: сборка матриц и их непосредственное использование. Сборка матрицы в свою очередь может быть разделена на этап вычисления шаблона матрицы (символьная сборка) и непосредственное вычисление коэффициентов матрицы (числовая сборка).

На этапе использования матрицы основной операцией является умножение матрицы на вектор, где наиболее эффективным является CSR-формат.

В случае использования неструктурированных сеток этап символьной сборки является нетривиальной операцией и сводится к неупорядоченному добавлению новых элементов в шаблон матрицы. Как было отмечено ранее, такая операция в случае использования CSR формата неэффективна.

Поэтому часто для этапов сборки и расчёта используют разные форматы хранения матриц, первый из которых оптимизирован для операции вставки, а второй – для операции умножения на вектор. В качестве формата, оптимизированного для вставки, можно представить формат массива словарей (List of dictionaries), где каждая строка матрицы описывается словарём, ключём которого является индекс колонки, а значением – величина соответствующего матричного коэффициента.

С использованием синтаксиса C++ такой формат может быть описан следующим образом:

```
std::vector<std::map<size_t, double>> data;
```

Матрица вида (D.20) в таком формате примет вид

```

data = {
    {0: 2.0, 3: 1.0},
    {1: 3.0, 2: 5.0, 3: 4.0},
    {2: 6.0},
    {1: 7.0, 3: 8.0}
};

```

Добавление нового матричного коэффициента сведётся к вставке элемента в словарь:

```
data[i][j] = value;
```

А основной операцией для такого формата будет служить конверсия в CSR:

```
std::vector<size_t> addr{0};
std::vector<size_t> cols;
std::vector<double> vals;
for (size_t irow=0; irow < data.size(); ++irow){
    for (auto it: data[irow]){
        cols.push_back(it.first);
        vals.push_back(it.second);
    }
    addr.push_back(addr.back() + data[irow].size());
}
```

Поскольку данные в контейнере типа `std::map` итерируются в отсортированном по ключам порядке, то полученный в результате массив `cols` также является локально отсортированным.

D.3 Методы решения систем уравнений

Будем рассматривать систему

$$Au = f. \quad (\text{D.21})$$

D.3.1 Простые итерационные алгоритмы

D.3.1.1 Метод Якоби

Будем рассматривать систему уравнений вида

$$\sum_{j=0}^{N-1} A_{ij}u_j = r_i, \quad i = \overline{0, N-1}$$

относительно неизвестного сеточного вектора $\{u\}$.

В классическом виде алгоритм Якоби формулируется в виде

$$\hat{u}_i = \frac{1}{A_{ii}} \left(r_i - \sum_{j \neq i} A_{ij}u_j \right)$$

Произведём некоторые преобразования

$$\begin{aligned} \hat{u}_i &= \frac{1}{A_{ii}} \left(r_i - \sum_j A_{ij}u_j + A_{ii}u_i \right) \\ &= u_i + \frac{1}{A_{ii}} \left(r_i - \sum_j A_{ij}u_j \right) \end{aligned}$$

Таким образом, программировать итерацию этого алгоритма, обновляющую значения массива $\{u\}$, можно в виде

```
 $\check{u} = u;$   
for  $i = \overline{0, N-1}$   
     $u_i += \frac{1}{A_{ii}} \left( r_i - \sum_{j=0}^{N-1} A_{ij}\check{u}_j \right)$   
endfor
```

D.3.1.2 Метод Зейделя

Формулируется в виде

$$\hat{u}_i = \frac{1}{A_{ii}} \left(r_i - \sum_{j < i} A_{ij}\hat{u}_j - \sum_{j > i} A_{ij}u_j \right).$$

Поскольку этот метод неявный относительно уже найденных на итерации значений, то в отличие от метода Якоби этот алгоритм не требует создания временного массива $\hat{u}$ при программировании. Псевдокод для реализации итерации этого метода можно записать как

```

for  $i = \overline{0, N-1}$ 
     $u_i += \frac{1}{A_{ii}} \left( r_i - \sum_{j=0}^{N-1} A_{ij} u_j \right)$ 
endfor

```

D.3.1.3 Метод последовательных верхних релаксаций SOR

Этот метод основан на добавлении к решению результатов итераций Зейделя с коэффициентом $\omega > 1$. То есть он изменяет решение по тому же принципу, что и метод Зейделя, но искусственно увеличивает эту добавку.

Формулируется этот метод в виде

$$\hat{u}_i = (1 - \omega)u_i + \frac{\omega}{A_{ii}} \left(r_i - \sum_{j < i} A_{ij} \hat{u}_j - \sum_{j > i} A_{ij} u_j \right).$$

Для устойчивости метода необходимо $\omega < 2$. В частности, для одномерных задач, заданных на единичном отрезке, для оптимальной сходимости можно использовать соотношение $\omega \approx 2 - 5h$, где h – шаг сетки.

Итерация этого метода по аналогии с методом Зейделя может быть запрограммирована в виде

```

for  $i = \overline{0, N-1}$ 
     $u_i += \frac{\omega}{A_{ii}} \left( r_i - \sum_{j=0}^{N-1} A_{ij} u_j \right)$ 
endfor

```

D.3.2 Метод коррекции поправки

Пусть u^n – некоторое приближённое решение системы (D.21) на итерационном слое. Введём невязки как

$$r^n = f - (Au)^n \quad (\text{D.22})$$

и поправку для перехода на следующий итерационный слой:

$$\Delta u^n = u^{n+1} - u^n.$$

Обобщённый процесс с предобуславливателем В для исходной системы можно записать в виде

$$B \Delta u^n = r^n \quad (\text{D.23})$$

Этот процесс является согласованным: если в качестве u^n взять точное решение системы (D.21), то $r^n = 0$, и следовательно, $\Delta u^n = 0$.

Алгоритм решения исходной системы будет иметь следующий вид:

1. задать начальное приближение u^n при $n = 0$
2. вычислить невязку r^n из (D.22)
3. если норма невязки $\|r^n\| < \epsilon$, то завершить итерации
4. решив систему (D.23) получить поправку $\Delta u^n = B^{-1}r^n$.
5. найти следующее приближение $u^{n+1} = u^n + \Delta u^n$
6. увеличить счётчик $n = n + 1$ и перейти на шаг 2.

Конкретный итерационный метод целиком определяется выбором предобуславливателя. В качестве предобуславливателя обычно выбирают матрицу, похожую на A , но при этом легкообратимую. Некоторые примеры:

- если выбрать константу $B = 1/\gamma$, то получим простой итерационный процесс Рундсона с шагом γ .
- диагональная часть исходного оператора $B = \{a_{ii}\}$ приводит к методу Якоби п. D.3.1.1
- нижняя треугольная часть исходной матрицы $B = \{a_{ij}\}, j \leq i$ даёт метод Зейделя п. D.3.1.2.

Если в качестве предобуславливателя взять исходную матрицу A , то итерационный процесс сойдётся за одну итерацию. Действительно из (D.22), (D.23):

$$u^{n+1} = u^n + A^{-1}(f - Au^n) = A^{-1}f$$

Это метод требует прямого обращения только предобуславливателя B , но не исходного оператора A , поэтому он может быть применён в том числе и для решения нелинейных систем.

Для обращения самого предобуславливателя в свою очередь так же может быть использован итерационный алгоритм. Этот итерационный процесс будет являться внутренним, поэтому не будет требовать глубокой сходимости. Зачастую для обращения предобуславливателя достаточно сделать 2-3 поверхностные итерации одним из простых методов из п. D.3.1.

D.3.3 Методы подпространств Крылова

D.3.3.1 Итерационный процесс BiCGStab с предобуславливателем

Инициализация

1. есть начальное приближение u^0
2. вычисляем невязку

$$r_0 = f - Au_0$$

3. предобуславливаем невязку

$$r'_0 = M^{-1} r_0$$

4. запоминаем начальную невязку

$$\tilde{r} = r'_0$$

5. инициализируем

$$p'_0 = v'_0 = 0, \quad \rho_0 = \omega_0 = \alpha_0 = 1.0$$

Далее на k -ой итерации

1. проецируем текущую невязку на начальную

$$\rho_k = \tilde{r}^T \cdot r'_{k-1}$$

2. коэффициент β :

$$\beta_k = \frac{\rho_k}{\rho_{k-1}} \frac{\alpha_{k-1}}{\omega_{k-1}}$$

3. Обновляем направление

$$p'_k = r'_{k-1} + \beta_k (p'_{k-1} - \omega_{k-1} v'_{k-1})$$

4.

$$v'_k = M^{-1} A p'_k$$

5. коэффициент α

$$\alpha_k = \frac{\rho_k}{\tilde{r}^T \cdot v'_k}$$

6. Промежуточный вектор s :

$$s'_k = r'_{k-1} - \alpha_k v'_k$$

7. Промежуточный вектор t

$$t'_k = M^{-1} A s'_k$$

8. множитель ω

$$\omega_k = \frac{(t'_k)^T \cdot s'_k}{(t'_k)^T \cdot t'_k}.$$

9. обновляем невязку и решение

$$u_k = u_{k-1} + \alpha_k p'_k + \omega_k s'_k,$$

$$r'_k = s'_k - \omega_k t'_k$$

(D.24)

Е Работа с инфраструктурой проекта CFDCourse

В настоящем параграфе будут даны инструкции для разворачивания инфраструктуры сборки проекта для операционных систем Linux(Ubuntu), MacOS и Windows.

В процессе настройки будет необходимо установить систему контроля версий **Git**, систему контейнеризации **Docker** и (опционально) интегрированную среду разработки. Проект позволяет работать в любой среде разработки. Ниже будут приведены инструкции для настройки **vscode**.

Процесс сборки и запуска программ будет осуществляться в системе, развёрнутой в докере на основе Ubuntu 24.04. В дальнейшем систему, установленную непосредственно на компьютере, будем называть хост системой. А систему, развёрнутую в докере – контейнером.

Для успешной установке на хосте должно быть около 5Гб свободного места.

Е.1 Клонирование

Для клонирования проекта на локальный компьютер необходимо установить систему контроля версий Git и открыть терминал на хост-системе в папке, в которой планируется хранить папку с репозиторием. В нижеследующих инструкциях в качестве такой папки будет использоваться домашняя папка пользователя.

Ubuntu

Откройте терминал и в нём

```
sudo apt install git # установка гита
cd ~                 # переходим в папку,
                     # где будет храниться папка с репозиторием
```

Windows

В Windows необходимо скачать и установить дистрибутив <https://github.com/git-for-windows/git/releases/download/v2.51.0.windows.1/Git-2.51.0-64-bit.exe>.

Далее откройте командную строку `cmd`. И в ней перейдите в целевую папку

```
cd %USERPROFILE%
```

MacOs

Откройте терминал и в нём

```
# Установите Homebrew, если у вас его ещё нет
/bin/bash -c \
"$ (curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh) "
# установка гита
brew install git
# переходим в папку где будет храниться папка с репозиторием
cd ~
```

Далее необходимо клонировать репозиторий

```
git clone https://github.com/kalininei/CFDCourse27
```

В результате в домашней папке должна появиться папка с `CFDCourse27`.

Е.2 Разворачивание контейнера

Установим докер

Ubuntu+apt

Запустить скрипт, написанный на основе инструкций с официального сайта <https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>:

```
cd ~/CFDCourse27    # перейдём в репозиторий
./scripts/ubuntu_docker_install.sh
```

Windows/MacOs+DockerDesktop

Скачайте и установите дистрибутив с официального сайта

- <https://docs.docker.com/desktop/setup/install/windows-install/> для Windows
- <https://docs.docker.com/desktop/setup/install/mac-install/> для MacOS

Далее следуйте процедуре установки десктопного приложения. После установки запустите `DockerDesktop`. Этап регистрации при запуске опционален и может быть пропущен.

Далее в терминале находясь в директории `CFDCourse27` развернём контейнер:

```
docker compose up --build -d
```

Если вы работаете на Unix-системе с несколькими пользователями и ваш пользователь не является дефолтным, то лучше собрать контейнер под своим пользовательским id. Для этого нужно определить необходимые переменные окружения выполнив вместо последней команды из предыдущего листинга:

```
USER_ID=$(id -u) GROUP_ID=$(id -g) docker compose up --build -d
```

Альтернативно можно перестраивать контейнер, запуская с хост системы скрипт `scripts/rebuild_container.sh`.

На этом этапе будут скачаны необходимые образы и запущен контейнер. По окончании можно убедиться, что контейнер работает

```
docker ps
```

В случае работы с `DockerDesktop` запущенный контейнер будет виден в графическом интерфейсе во вкладке `Containers`.

Е.3 Базовая разработка

Чтобы скомпилировать проект необходимо войти в терминал контейнера. Далее

```
docker ps          # Убедимся, что контейнер cfd27 запущен
docker exec -it cfd27 bash # Войдём в него
```

На Unix системе можно использовать скрипт `scripts/attach_to_container.sh`.

Папка хоста с репозиторием `CFDCourse` примонтирована к папке контейнера `/app`. Войдём туда

```
cd /app # заходим в директорию
ls -alh # смотрим список файлов
```

Для удобства в контейнере установлен консольный файловый менеджер

`mc`, который можно использовать для операций с файлами. Так же есть его псевдоним

`mcs`, который отличается от базового `mc` тем, что запоминает текущую директорию при выходе.

Благодаря чему его можно использовать для навигации вместо `cd`.

Е.3.1 Особенности проекта

- Проект состоит из статически линкуемой библиотеки `libcfd27.a` и исполняемого файла `cfd27_test` с тестовыми программами для этой библиотеки. Исходники библиотеки лежат в директории `src/cfd`, исходники тестов – `src/test`,
- Используется 20-ый стандарт C++,
- Сборка осуществляется в системе `cmake`,
- Для написания тестов используется фреймворк `Catch2`,
- В проекте установлены жёсткие правила работы с предупреждениями компиляции, из-за которых они как обрабатываются ошибки,
- В проекте установлены форматтеры для исходных кодов на C++, python, `cmake`. При сохранении любого исходного файла этих форматов в `vscode` форматтер будет вызван автоматически. Если исходники модифицировались иначе, то запустить форматтер для всех файлов проекта можно скриптом `scripts/formatall.sh` из папки `/app`.

Е.3.2 Сборка в отладочном режиме

Из папки `/app` контейнера:

```
mkdir build          # создаём папку, в которую будут строиться программа
cd build             # Заходим
cmake .. -DCMAKE_BUILD_TYPE=Debug # Строим сборочные скрипты
make -j4             # Компилируем на 4-х потоках
```

Сама папка

`build` является временной папкой для построения. Она не подключена к системе контроля версий. И может быть удалена и создана заново (например для полной гарантированной очистки кэша построения). Бинарные файлы будут построены в папку `/app/build/bin`. Для запуска тестов зайдём в эту папку:

```
cd bin
./cfd27_test          # запуск всех тестов
./cfd27_test [grid1]  # запуск единственного тест кейса
```

Е.3.3 Сборка в релизном режиме

Отличается от сборки в отладочном режиме только флагом `cmake`. Будем строить в папку `build_release`. Также от папки `/app`

```
mkdir build_release          # создаём папку
cd build_release             # Заходим
cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo # Или просто Release,
                                # если отладочная информация не нужна
make -j4                     # Компилируем на 4-х потоках
cd bin                       # заходим в папку с исполняемым файлом
./cfd27_test                 # запуск всех тестов
./cfd27_test [grid1]        # запуск единственного тест кейса
```

Е.3.4 Работа с кодом

Работать с исходными файлами можно находясь на хосте и используя любой удобный текстовый редактор. (например

`notepad++` на Windows). Порядок работы будет выглядеть следующим образом

- Редактировать исходные файлы на хосте в папке `CFDCourse27/src`
- Для сборки переключиться на терминал и выполнить вышеописанную процедуру сборки
- Если сборка не прошла из-за ошибок в коде, эти ошибки будут распечатаны в терминал с указанием файла и номера строки
- Для отладки можно использовать консольный отладчик `gdb` из терминала

Е.4 Разработка в vscode

Е.4.1 Подключение к контейнеру

Необходимо установить vscode на хосте следуя инструкциям с официального сайта <https://code.visualstudio.com/Download>. В самом vscode нужно установить расширение

`Remote Explorer, Dev Containers` для удалённой разработки в контейнере. Далее нужно переключиться на вкладку RemoteExplorer (см. рис. 19).

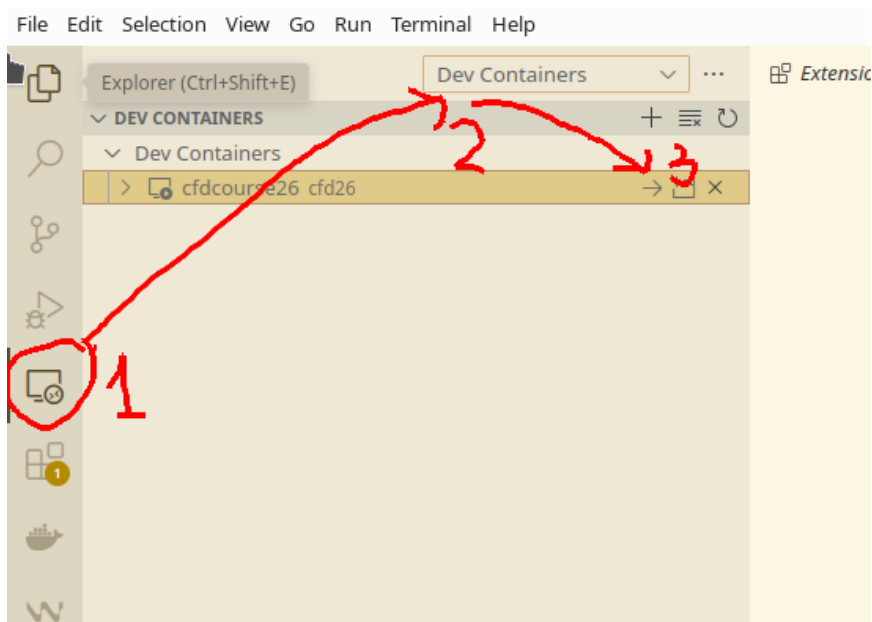


Рис. 19: Подключения vscode к контейнеру

Е.4.2 Настройки vscode

Далее необходимо открыть папку `/app`:

File->Open Folder... Контейнер содержит в себе базовые настройки vscode, которые при сборке контейнера копируются в папки `.vscode`, `.vscode-server`.

Следует отметить, что удалённо подключённый vscode не может пользоваться расширениями, установленными на хосте. Все необходимые расширения нужно устанавливать в контейнер заново. Список базовых расширений, необходимых для работы, уже содержится в настроечных файлах. Когда вы откроете папку `app` в vscode, вам будет предложено эти расширения установить. Нужно согласиться.

Настроечные папки

`.vscode`, `.vscode-server` игнорируются системой контроля версий и расположены на хосте. То есть вы можете дополнительно установить туда любые свои расширения и донстроить vscode как вам удобно. Эти настройки не будут зависеть от ветки гита и не будут затираться при пересборке контейнера.

Если всё же понадобится обнулить настройки vscode, нужно

1. удалить папки `.vscode`, `.vscode_server`
2. удалить все настройки из конфигурационного файла контейнера (`ctrl+shift+p`, `open container configuration file`)
3. выйти из контейнера на vscode: **File->Close Remote Connection**
4. остановить и пересобрать контейнер на хосте

```
docker stop cfd27
docker compose up --build -d
```

Е.4.3 Сборка и отладка

В папке `.vscode` лежат базовые таски и лаунчи для компиляции и запуска программы. В случае необходимости можете дополнить базовый набор своими командами. Согласно настройкам по умолчанию по нажатию `F5` происходит сборка программы в отладочном режиме и запуск отладки тестовой программы с прогоном всех тестов. Посмотреть результаты можно на вкладке `TERMINAL`. В случае ошибок компиляции, список этих ошибок будет виден на вкладке `PROBLEMS`.

Для отладки конкретного теста необходимо запустить тестовую программу с аргументом (например `cfid27_test [grid1]`). Чтобы передать программе аргумент нужно этот аргумент прописать в файле `.vscode/launch.json` в поле `args`. Либо создать ещё одну конфигурацию запуска с вашими аргументами и указать эту конфигурацию в настройке `Run And Debug`.

Чтобы собрать программу без запуска нужно выполнить таск (`ctrl+shift+b`):

`cmake: build debug`, `cmake: build release` для отладочного и релизного режима соответственно.

В целом сборка на `vscode` представляет из себя автоматизированный алгоритм, представленный в п. [Е.3](#). То есть исполняемая программа в дебаговой версии кладётся в папку `build`, в релизной – в `build_release` и её можно запустить из терминала. Удаление этих папок ведёт к полной очистке кэша построения.

Е.5 Работа с системой контроля версий

Работать с гитом можно как с хоста (из папки `CFDCourse27`), так и из контейнера (из папки `\app`). Ниже будут даны инструкции для работы с гитом в консоли. Альтернативно, можно установить графический интерфейс (например `GitExtensions` для Windows) или командами `vscode` на вкладке `Source Control`.

Из системы контроля версий исключены следующие каталоги:

- `build*/` – папки со сборками,
- `.vscode`, `.vscode-server` – настройки и расширения `vscode`,
- `local_data` – папка для хранения любых пользовательских данных.

Изменения из этих папках не будут отслежены и скоммичены.

Е.5.1 Порядок работы с репозиторием `CFDCourse`

Основная ветка проекта –

`main`. После каждой лекции в эту ветку будет отправлен коммит с сообщением `lect{index}`. В этом коммите будет дополнен pdf документ с содержанием лекции, задание по итогам лекции и необходимые для этого задания изменения в коде.

Е.5.1.1 Получение последнего коммита

Таким образом, **после лекции**, после того, как изменение

`lect{index}` придёт на сервер, необходимо выполнить следующие команды

```
git checkout main      # перейти на основную ветку
git pull               # получить изменения
```

Если изменения не содержали никаких изменений в настройках контейнера `Dockerfile`, `docker-compose.yaml`, то для сборки проекта рестарт контейнера не требуется. Иначе требуется пересобрать контейнер:

```
docker stop cfd27      # остановить текущий контейнер
docker compose up --build -d # пересобрать новый
```

Е.5.1.2 Создание коммита с текущим дз

После того, как была сделана какая то работа по заданиям,

```
git checkout -b hw-lect{index}  # создать локальную ветку, содержащую задание
git add .
git commit -m "{свой комментарий}" # скоммитить свои изменения в эту ветку
```

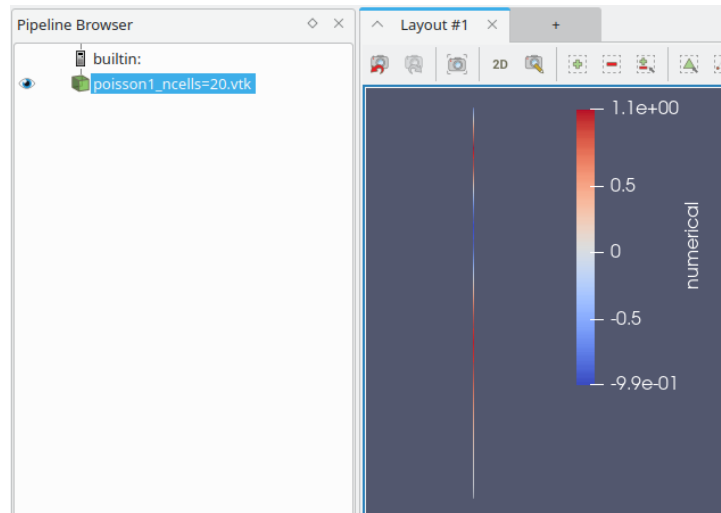
Даже если задание выполнено не до конца, вы в любой момент можете переключиться на ветку с заданием и его доделать

```
git checkout hw-lect{index}
```

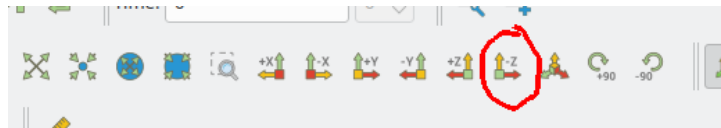
Е.6 Paraview

Е.6.1 Данные на одномерных сетках

Заданные на сетке данные паравью показывает цветом. Поэтому при загрузке одномерных сеток можно видеть картинку типа

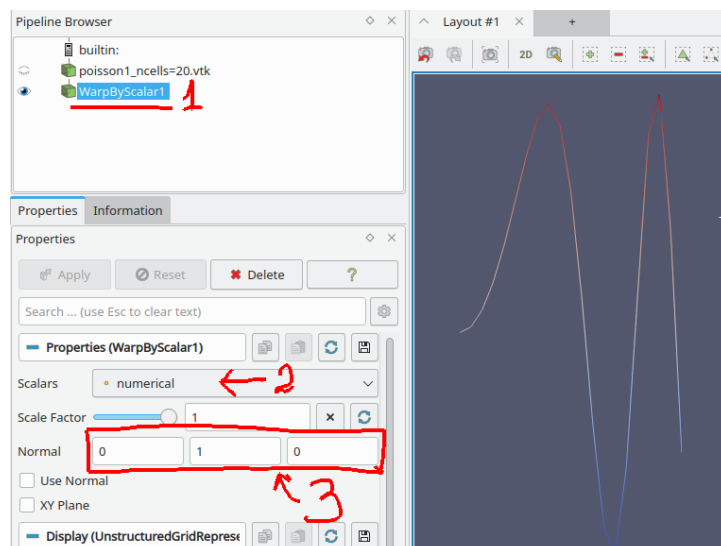


Развернуть изображение в плоскость ху



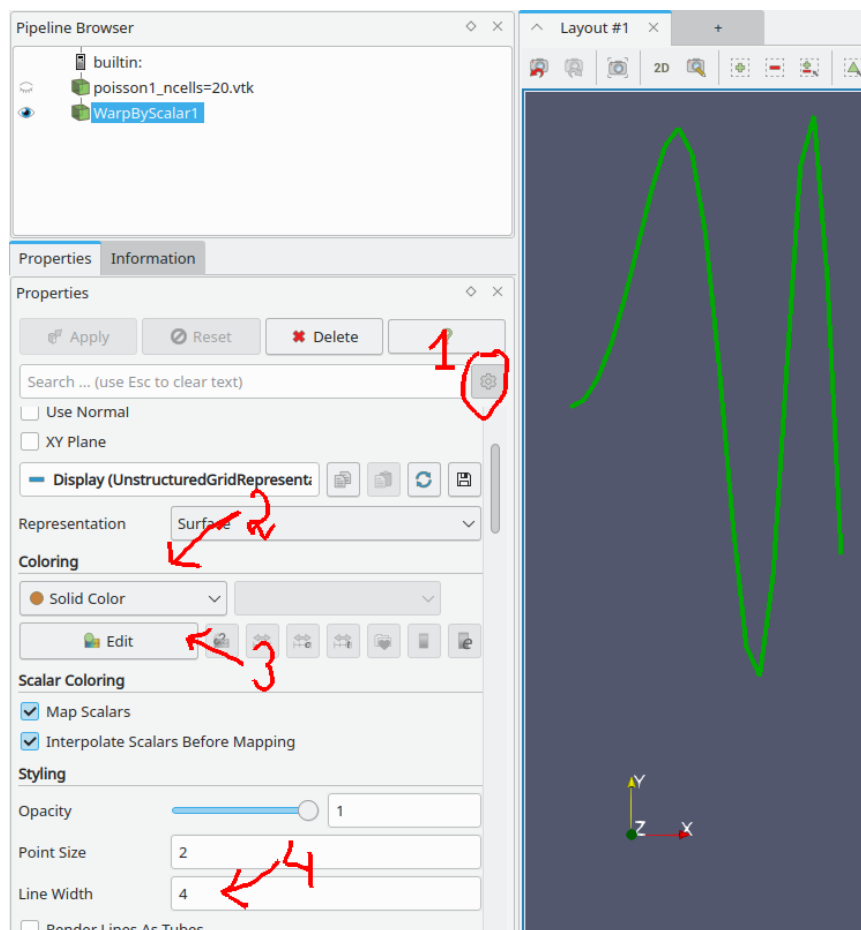
Отобразить данные в виде у-координаты Для того, что бы данные отображались в качестве значения по оси ординат, к загруженному файлу необходимо

1. применить фильтр **WarpByScalar** (В меню **Filters->Alphabetical->Warp By Scalar**)
2. в меню настройки фильтра указать поле данных, для отображения (numerical в примере ниже)
3. И настроить нормаль, вдоль которой будут проецироваться данные (в нашем случае ось у)



Цвет и толщина линии

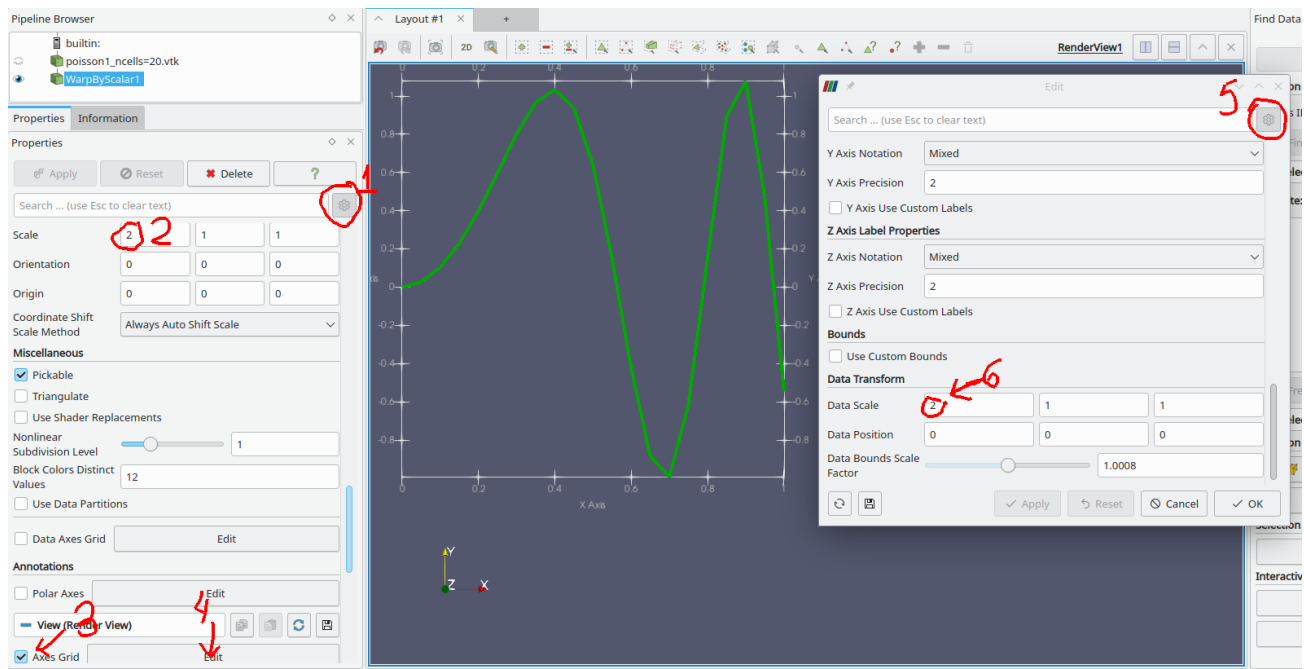
1. Включить подробные опции фильтра
2. Сменить стиль на **Solid Color**
3. В меню **Edit** выбрать желаемый цвет
4. В строке **Line Width** указать толщину линии



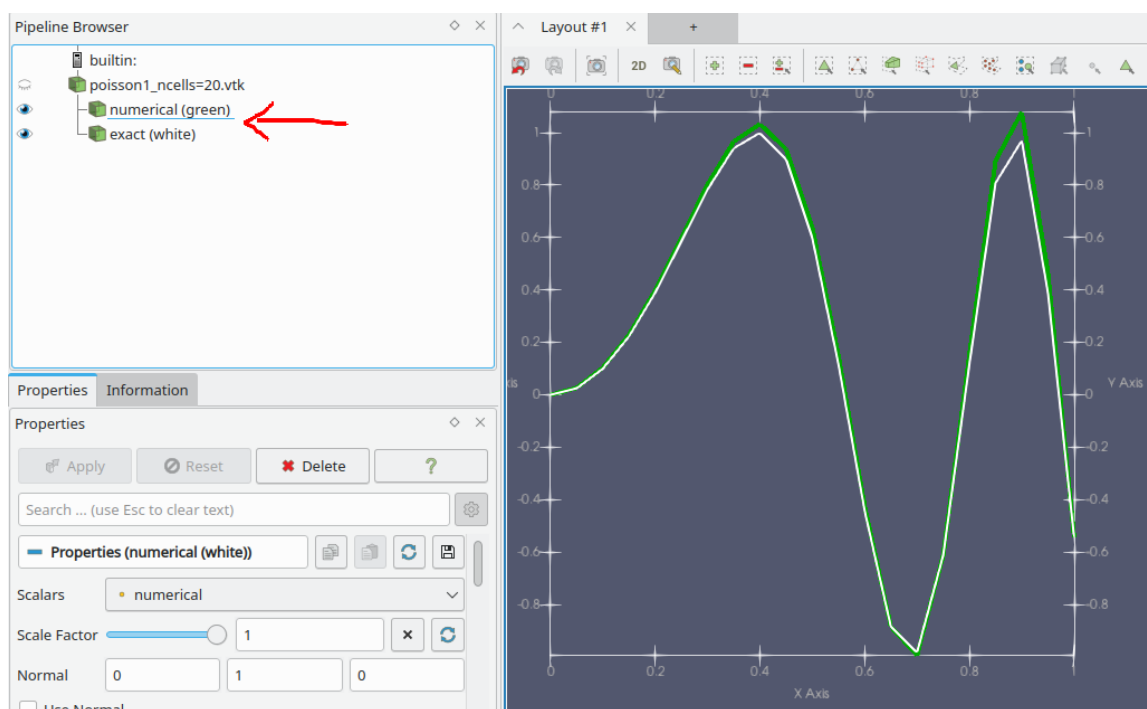
Настройка масштабов и отображение осей координат

1. Отметьте подробные настройки фильтра
2. В поле **Transforming/Scale** Установите желаемые масштабы (в нашем случае растянуть в два раза по оси x)
3. Установите галку на отображение осей
4. откройте меню натройки осей
5. В нём включите подробные настройки
6. И также поставьте растяжение осей

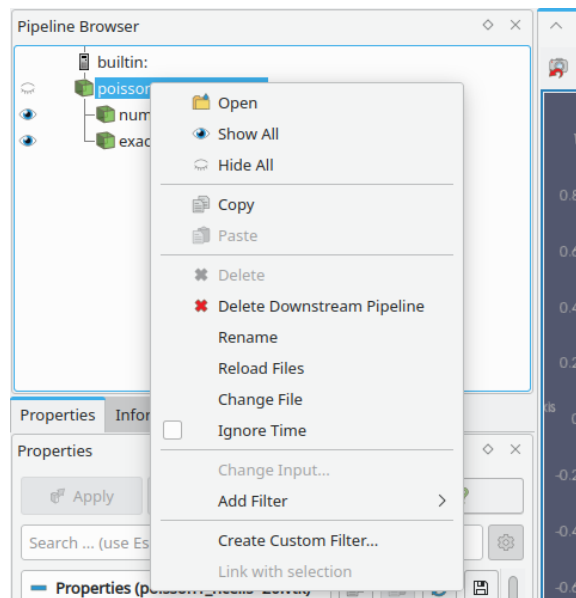
В случае, если масштабировать график не нужно, достаточно выполнить шаг 3.



Построение графиков для нескольких данных Если требуется нарисовать рядом несколько графиков для разных данных из одного файла, примените фильтр **Warp By Scalar** для этого файла ещё раз, изменив поле **Scalars** в настройке фильтра. Для наглядности измените имя узла в Pipeline Browser на осмысленные



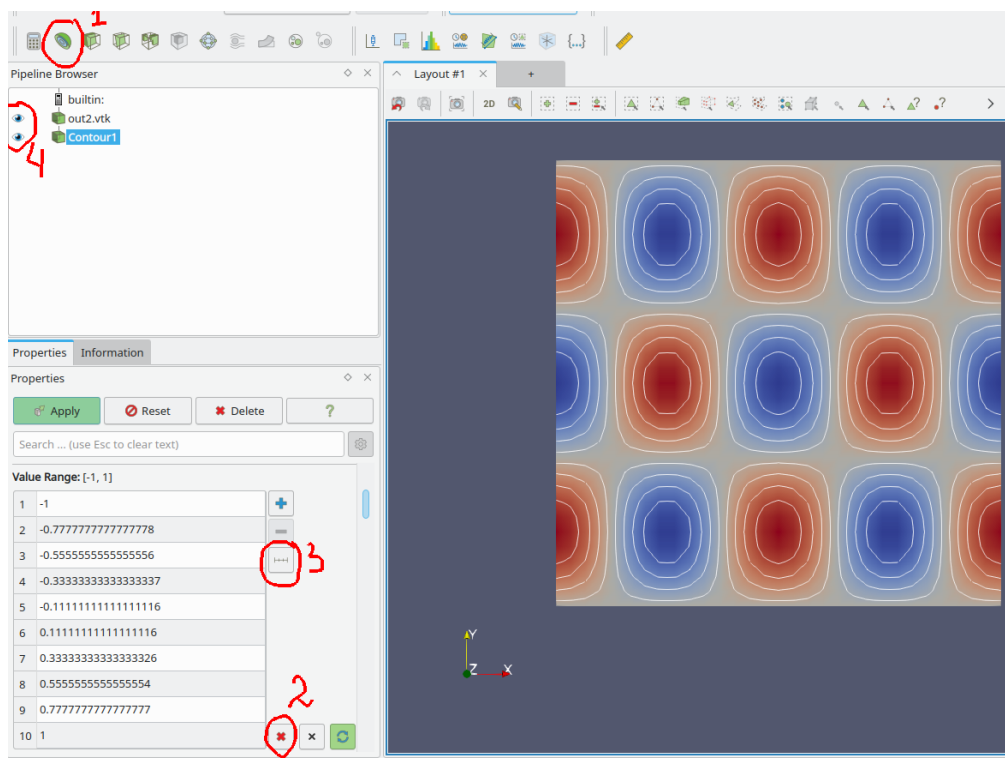
Обновление данных при изменении исходного файла В случае, если исходный файл был изменён, нужно в контекстном меню узла соответствующего файла выбрать **Reload Files** (или нажать F5). Если те же самые фильтры нужно применить для просмотра другого файла нужно в этом меню нажать **Change File**.



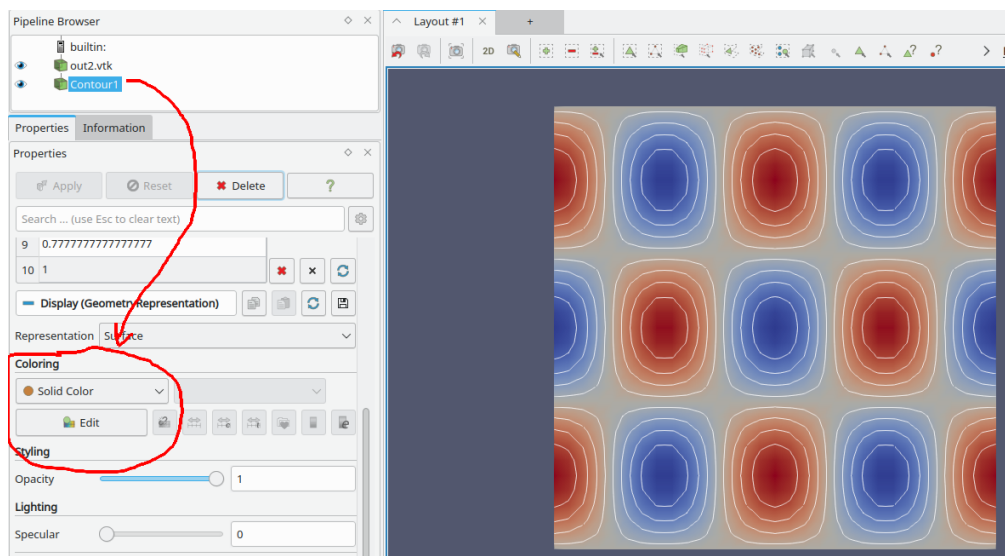
Е.6.2 Изолинии для двумерного поля

Ниже представлен алгоритм отрисовки изолиний для данных не сетке, на которой решение известно в узлах. Если вы работаете с данными в ячейках (конечнообъёмная сетка), следует предварительно применить фильтр “Cell Data To Point Data”.

1. Нажмите иконку **Contour** (или **Filters/Contour**) В настройках фильтра Contour by выберите данные, по которым нужно строить изолинии.
2. В настройках фильтра удалите все существующие записи о значениях для изолиний
3. Добавьте равномерные значения. В появившемся меню установите необходимое количество изолиний и их диапазон.
4. Если необходимо, включите одновременное отображения цветного поля и изолиний.



Задание цвета и толщины изолинии В случае, если нужно сделать изолинии одного цвета, установите поле **Coloring/Solid color** в настройках фильтра. Там же в меню **Edit** можно выбрать цвет. Для установления толщины линии включите подробные настройки и найдите там опцию **Styling/Line Width**.

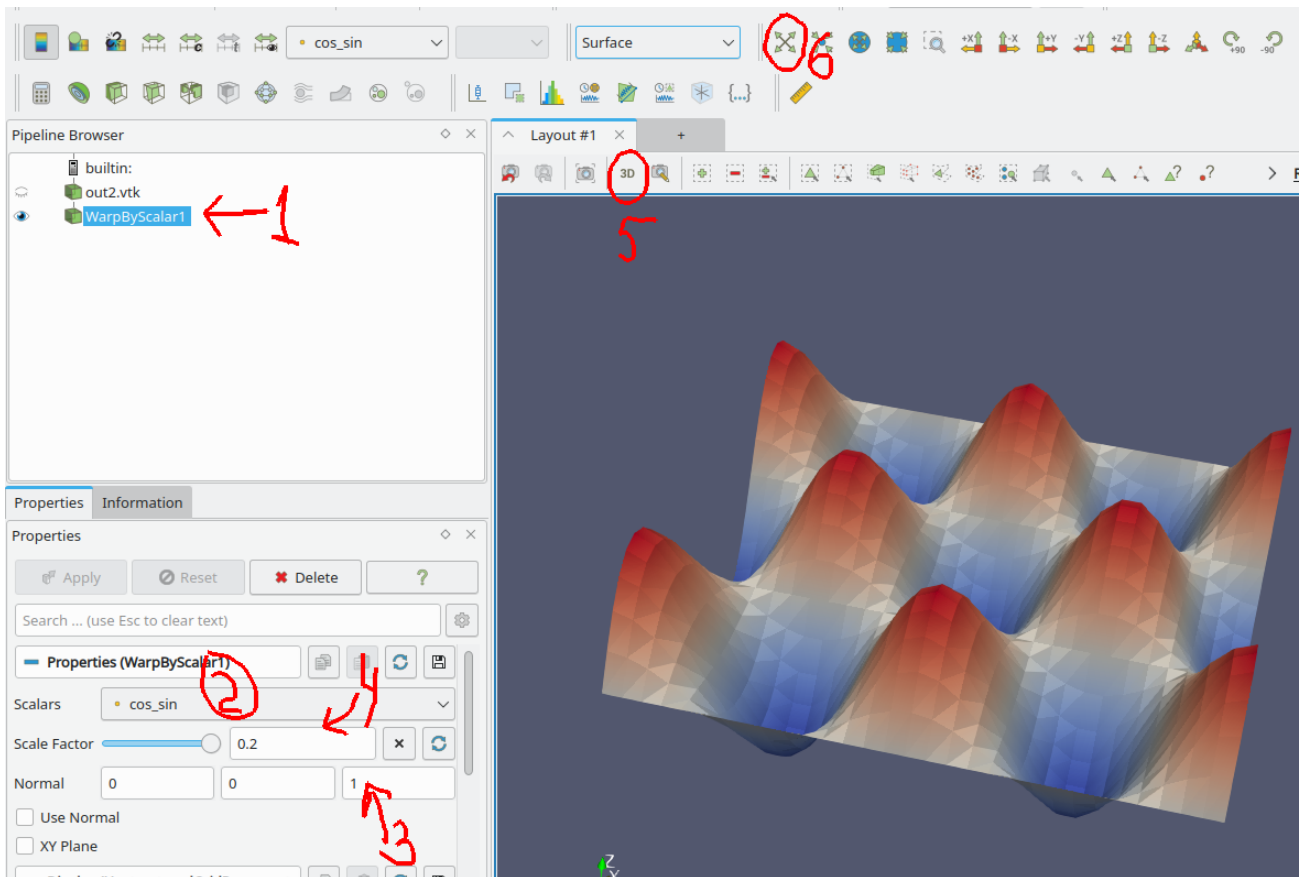


Е.6.3 Данные на двумерных сетках в виде поверхности

По аналогии с одномерным графиком (п. Е.6.1), двумерные поля так же можно отобразить, проектируя данные на геометрическую координату для получения объёмного графика. Для этого

1. Включите фильтр **Filters/Warp By Scalar**
2. В настройках фильтра установите данные, которые будут проектироваться на координату z
3. Установите нормаль для проецирования (ось z)

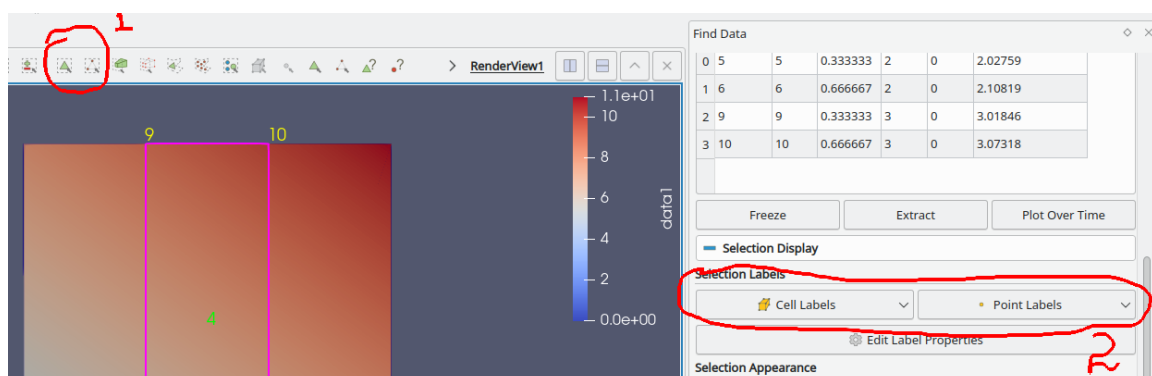
4. Если нужно, выберите масштабирования для этой координаты
5. После нажатия **Apply** включите трёхмерное отображение
6. Если данные не видно, обновите экран.



Е.6.4 Числовых значения в точках и ячейках

Иногда в процессе отладки или анализа результатов расчёта требуется знать точное значение поля в заданном узле или ячейке сетки. Для этого

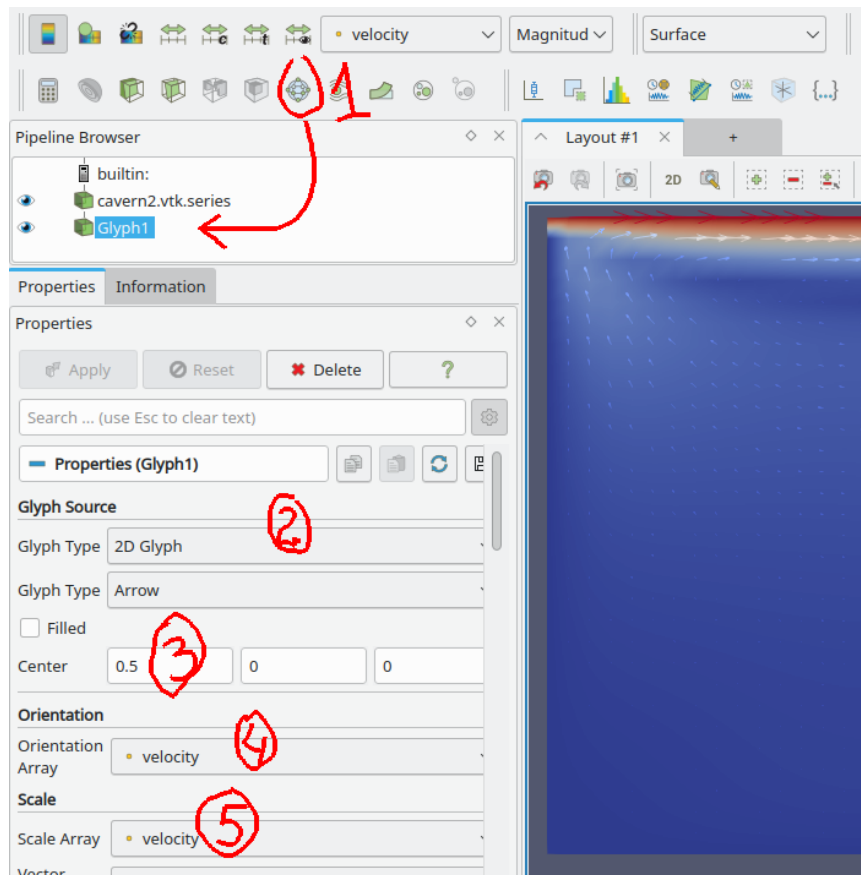
1. Включить режим выделения точек или ячеек (иконка (1 на рисунке) или горячие клавиши **s**, **d**). Выделить мышкой интересующую область
2. В окне **Find data** (или **Selection Inspector** для старых версий Paraview) отметить поле, которое должно отображаться в центрах ячеек и в точках (2 на рисунке). Если такого окна нет, включить его из основного меню **View**.



Е.6.5 Векторные поля

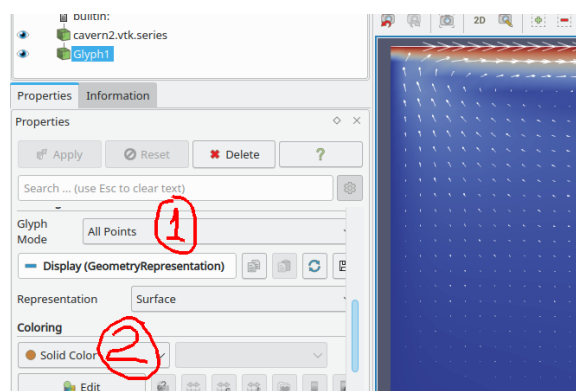
Открыть файл `vtk` или `vtk.series`, который содержит векторное поле. Далее

1. Создать фильтр **Glyph**
2. Задать двумерный тип стрелки
3. Сместить центр стрелки, чтобы она исходила из точки, к которой приписана
4. Отметить необходимое векторное поле в качестве ориентации
5. Отметить необходимое векторное поле для масштабирования Нажать **Apply**.



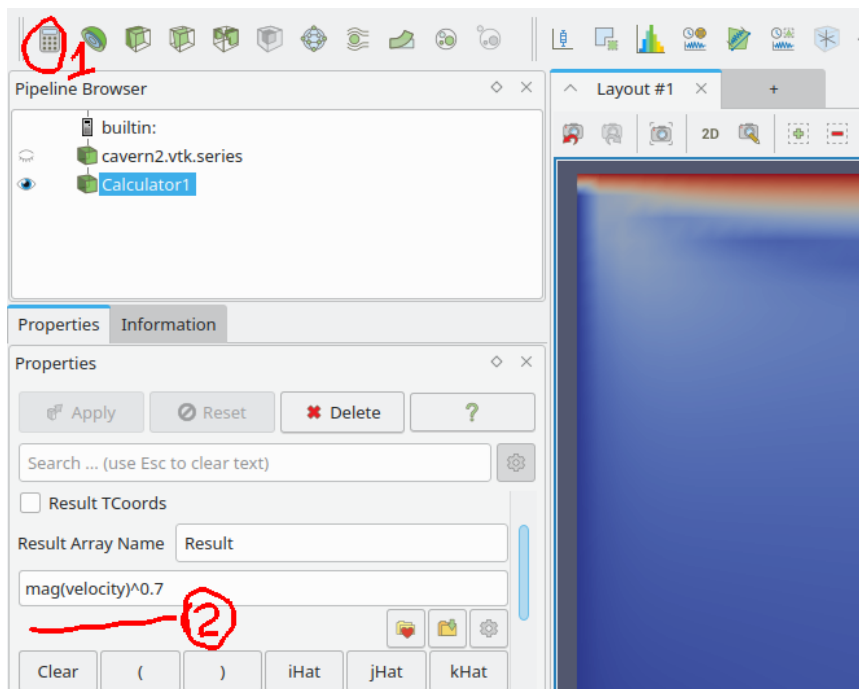
Настройка отображения стрелок

1. Выбрать необходимый **Glyph-mode**. Если сетка небольшая, то можно **All Points**.
2. Установить белый цвет для стрелок. Нажать **Apply**.



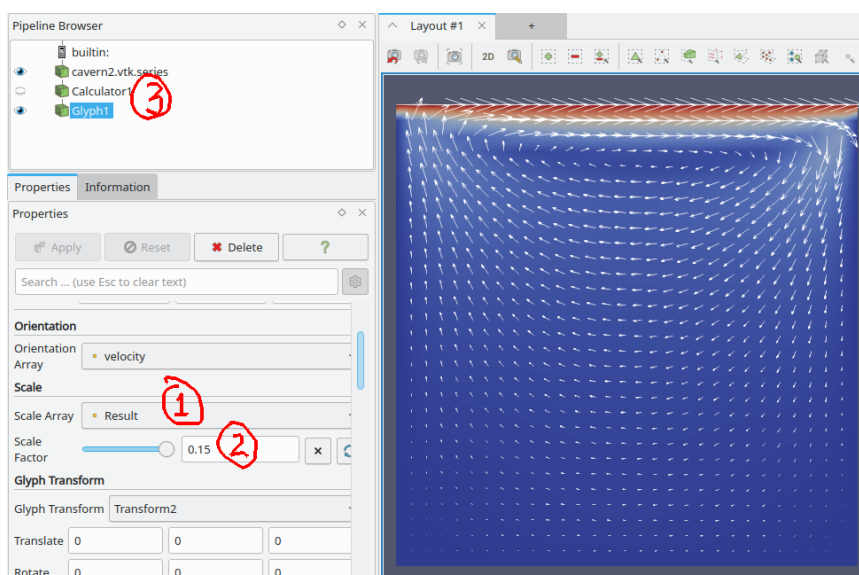
Уменьшения разброса по длине стрелок Если разброс по длинам стрелок слишком велик, его можно подравнять, введя новую функцию $|\mathbf{v}|^\alpha$ – длина вектора в степени меньше единицы (например, $\alpha = 0.7$). Такую функцию можно создать через калькулятор

1. Начиная от загруженного файла создать фильтр **Calculator**
2. Там вбить необходимую формулу



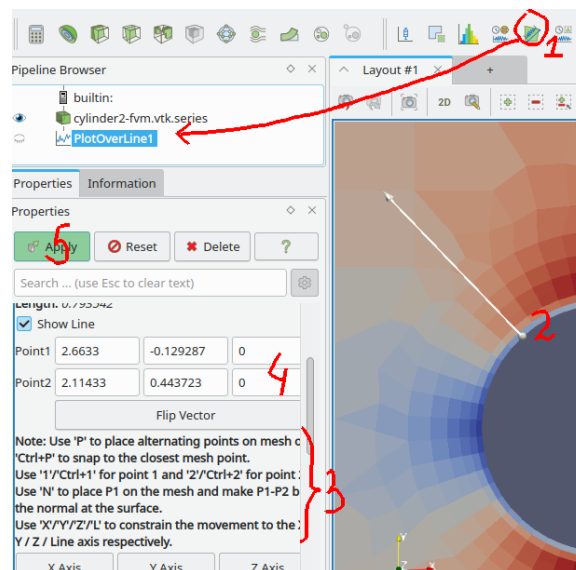
Созданную функцию нужно прокинуть в **Glyph** в качестве коэффициента масштабирования

1. В **Scale Array** фильтра **Glyph** указать уже результат работы **Calculator**-а (**Result** по умолчанию),
2. Подтянуть значение **Scale Factor** до приемлимого
3. Не забыть отключить вспомогательное поле **Calculator** из отображения



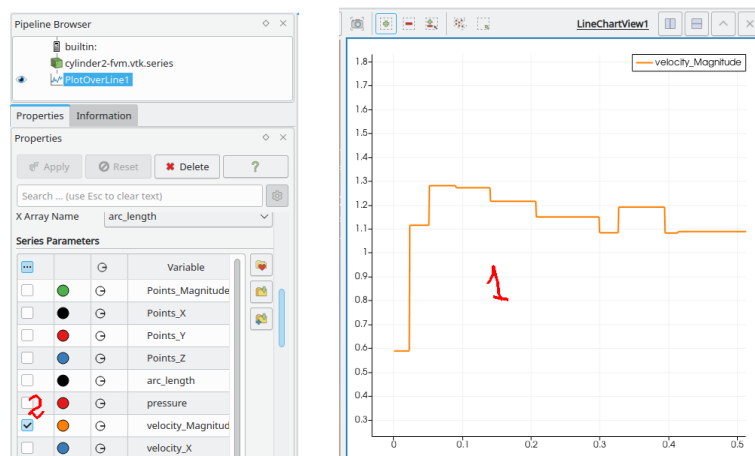
Е.6.6 Значение функции вдоль линии

1. Выбрать фильтр **Plot Over Line** иконкой или в меню **Filters**
2. Установить начальную и конечную точку сечения
3. Можно использовать привязку к узлам сетки с помощью горячих клавиш (в подсказках написано)
4. Можно установить координаты руками в соответствующем поле. Для двумерных задач проследить, что координата Z равна нулю
5. Нажать **Apply**



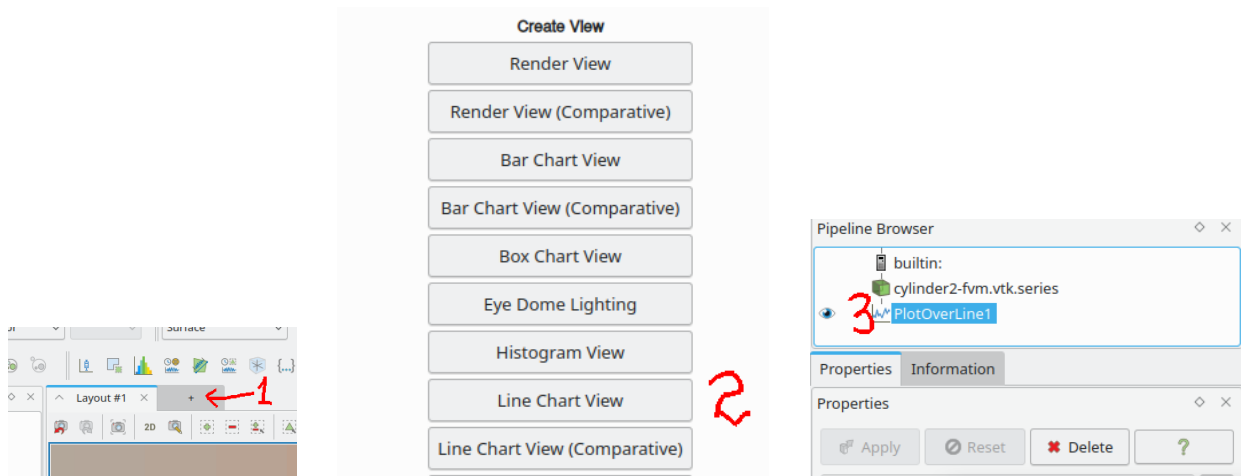
Настройка графика

1. После установок появится дополнительное окно типа `Line Chart View` с нарисованным графиком.
2. Сделав это окно активным в настройках фильтра `PlotOverLine` можно выбрать, какие поля рисовать (`Series Parameters`)



Отрисовка в отдельном окне

1. Открыть новую вкладку
2. Выбрать **Line Chart View**
3. Выбрать предварительно созданный фильтр с одномерным графиком



Е.7 Hybmesh

Генератор сеток на основе композитного подхода. Работает на основе python-скриптов. Полная документация <http://kalininei.github.io/HybMesh/index.html>

Е.7.1 Построение сеток

Скрипты построения сетки лежат в папке `/app/test-data`. Для запуска скрипта построения сетки следует находясь в контейнере перейти в эту папку и запустить (например для `trigrid.py`)

```
cd /app/test-data  
hybmesh -sx trigrid.py
```

При первом запуске система предложит собрать и установить `hybmesh` из исходников. Следует согласиться, введя `y`. Первое построение может занять несколько минут.

Е.8 Приёмы отладки

Е.8.1 Печать разреженной матрицы

Для вывода разреженных матриц любого формата (реализующего интерфейс `ISparseMatrix`) Можно воспользоваться функцией `dbg::print` (её можно вызывать непосредственно в процессе отладки не прерываясь на расстановку нужных вызовов в коде и перекомпиляцию).

```
#include "cfd/debug/printer.hpp"
...

LodMatrix M(3);
M.add_value(0, 0, 1.0);
M.add_value(1, 0, 0.0);
M.add_value(1, 1, 4.0);
M.add_value(2, 2, 3.0);
dbg::print(M);
```

В результате в терминал выведется матрица

```
-- SIZE = 3x3
  1      *      *
  0      4      *
  *      *      3
```

Здесь `*` – отмечены места, где отсутствует значение в шаблоне матрицы (то есть ноль по построению). Заданные в шаблоне значения тоже могут быть нулями (как в настоящем примере на второй строке первой колонки).

Если матрица имеет большую размерность, то печатать всю матрицу целиком неудобно. Но можно воспользоваться печатью отдельной строки. Так вызов

```
dbg::print(1, M);
```

выведет только значения второй строки матрицы, находящиеся в шаблоне

```
-- ROW = 1
[  0] 0  // M[1, 0] = 0
[  1] 4  // M[1, 1] = 4
```

Е.8.2 Сохранение отладочных полей решения

В процессе отладки работы с сеточными данными часто возникает необходимость быстро отобразить их на сетке. Для этого в файле `debug/saver.hpp` имеется набор вспомогательных функций (каждая из них может быть запущена непосредственно из отладчика):

```
// скаляр в узлах сетки
void save_point_data(const IGrid& grid, const std::vector<double>& data);
```

```

// скаляр в ячейках сетки
void save_cell_data(const IGrid& grid, const std::vector<double>& data);
// вектор в ячейках сетки
void save_cell_vector(const IGrid& grid, const std::vector<Vector>& vec);
// скаляр в расширенных точках коллокации (ячейки + граничные грани)
void save_extended_colloc_data(const IGrid& grid, const std::vector<double>& data);
// скаляр в гранях сетки
void save_face_data(const IGrid& grid, const std::vector<double>& data);

```

Все эти функции сохраняют данные в файл `dbg.vtk` в текущую папку из которой запущена программа.

Е.8.3 Профилирование

Для профилирования (замера время исполнения участков кода) можно пользоваться набором глобальных функций, определённых в файле

`debug/tictoc.hpp`.

Простой замер времени Измерение времени исполнения участка кода:

```

#include "cfcd/debug/tictoc.hpp"
...

dbg::Tic("process1"); // начать замер (блок с именем process1)
// ... код, время исполнение которого требуется замерить
std::this_thread::sleep_for(std::chrono::milliseconds(40));
dbg::Toc("process1"); // окончить замер

```

По окончании работы программы в терминал будет распечатан отчёт о времени исполнения каждого блока (в миллисекундах)

----- Tictoc report (ms)			
Process	Total time	Calls count	Time per call

process1	40	1	40

Названия секций может быть произвольным и может повторяться. В последнем случае программа будет выводить как общее время исполнение этого блока, так и среднее время по запускам:

```

dbg::Tic("process1");
std::this_thread::sleep_for(std::chrono::milliseconds(40));
dbg::Toc("process1");

dbg::Tic("process1");
std::this_thread::sleep_for(std::chrono::milliseconds(50));
dbg::Toc("process1");

```

```

----- Tictoc report (ms)
Process          Total time    Calls count  Time per call
-----
process1          90              2           45

```

Замер времени внутри области видимости Чтобы не писать

Тос для остановки таймера можно использовать замер внутри текущей области видимости (внутри текущих фигурных скобок) используя метод **ScopedTic**. Например, чтобы замерить время работы метода можно написать

```

void heavy_func(){
    auto tic = dbg::ScopedTic("heavy_func");
    std::this_thread::sleep_for(std::chrono::milliseconds(50));

    // объект tic удалится при выходе из области видимости
    // и таймер для процесса heavy_func будет остановлен
}

```

Индексированные процессы Иногда бывает полезно разбить один процесс на несколько под-процессов. Для этого можно воспользоваться индексированными именами. В отчёт при этом пойдет как время исполнения каждого шага процесса, так и общее время.

```

{
    auto tic = dbg::ScopedTic("process", 1);
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
}
{
    auto tic = dbg::ScopedTic("process", 2);
    std::this_thread::sleep_for(std::chrono::milliseconds(20));
}
{
    auto tic = dbg::ScopedTic("process", 3);
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

```

```

----- Tictoc report (ms)
Process          Total time    Calls count  Time per call
-----
process          80              3           26
process-1        50              1           50
process-2        20              1           20
process-3        10              1           10

```

Это механизм удобно использовать для замера времени исполнения тела цикла (если кол-во итераций небольшое)

```
for (size_t i=0; i<3; ++i){
    dbg::Tic("loop", i);
    std::this_thread::sleep_for(std::chrono::milliseconds(50-i));
    dbg::Toc("loop", i);
}
```

Печать отчётов По умолчанию программа печатает финальный отчёт о выполнении по окончании работы программы. Если требуется распечатать отчёт раньше, можно явно вызывать метод `dbg::Report` или `dbg::FinReport`. Последний обнуляет все таймеры и может быть удобен при запуске нескольких расчётов с разными параметрами:

```
for (size_t n_cells: {10, 20, 50}){
    calculate(n_cells);
    FinReport(); // печать отчёта о профилировании только для текущего расчёта
}
```